

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Дисциплина: Системное программирование

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему

Программное средство «Музыкальный редактор»

БГУИР КП 6-05-0612-01 ПЗ 22

Выполнил: Наривончик А.М.
гр. 351004

Проверил: Базылев Е.Н.

Минск 2025

Учреждение образования
«Белорусский государственный университет информатики и
радиоэлектроники»

УТВЕРЖДАЮ
Заведующий кафедрой ПОИТ

(подпись)

2025 г.

ЗАДАНИЕ
по курсовому проектированию

Студенту Наривончик Александр Михайлович

1. Тема работы Программное средство «Музыкальный редактор»
2. Срок сдачи студентом законченной работы 11.12.2025 г.
3. Исходные данные к работе Платформа: разработка программного средства будет производиться на операционной системе Windows, используя язык программирования C++, библиотека функций WinAPI; для графического интерфейса применяется фреймворк Qt; Функциональность: программа должна давать возможности пользователю обрезать, склеивать аудио, накладывать аудиофайлы друг на друга, проигрывать их и экспортировать итоговый аудиофайл. Обработка данных: результатом работы программы должен быть экспортированный аудиофайл.
4. Содержание расчётно-пояснительной записки (перечень вопросов, которые подлежат разработке)
Введение. 1. Анализ литературных источников и формирование функциональных требований к разрабатываемому программному средству. 2. Проектирование и разработка программного средства. 3. Тестирование и проверка работоспособности программного средства. 4. Руководство по использованию программного средства. Заключение. Список использованных источников. Приложения

5. Перечень графического материала (с точным обозначением обязательных чертежей и графиков)

1. Схема алгоритма комбинирования двух аудиофайлов в один.

6. Консультант по курсовой работе

Базылев Е.Н.

7. Дата выдачи задания 7.09.2025 г.

8. Календарный график работы над проектом на весь период проектирования (с обозначением сроков выполнения и процентом от общего объема работы):

раздел 1,2 к 15.09.2025 – 15 % готовности работы;

разделы 3, 4 к 15.10.2025 – 30 % готовности работы;

разделы 5, 6 к 15.11.2025 – 60 % готовности работы;

раздел 7, 8 к 02.12.2025 – 90 % готовности работы;

оформление пояснительной записки и графического материала к 11.12.2025 – 100 % готовности работы.

Защита курсового проекта с 07 по 11 декабря 2025 г.

РУКОВОДИТЕЛЬ _____ Е.Н.Базылев
(подпись)

Задание принял к исполнению _____ А.М. Наривончик 7.09.2025 г.
(дата и подпись студента)

СОДЕРЖАНИЕ

Введение.....	5
1 Анализ литературных источников и формирование функциональных требований к разрабатываемому программному средству	6
1.1 Обзор существующих аналогов.....	6
1.1.1 Программное средство «FL Studio»	6
1.1.2 Программное средство «Audacity».....	7
1.1.3 Программное средство «GarageBand»	8
1.2 Описание средств разработки	8
1.2.1 Использование динамических структур	9
1.2.2 Использование файлов	9
1.3 Спецификация функциональных требований	9
2 Проектирование и разработка программного средства	11
2.1 Структура программы.....	11
2.2 Проектирование интерфейса программного средства.....	11
2.1.1 Главное окно	11
2.1.2 Окно редактирования сэмпла.....	12
2.1.3 Окно инструкции.....	13
2.1.4 Окно о приложении.....	13
2.3.1 Проигрывание аудиотрека	14
2.3.2 Сложение аудиоданных.....	15
2.3.3 Обрезка аудиофайлов	18
2.3.4 Сохранение проекта	19
2.3.5 Экспорт аудиофайла	20
3 Разработка программного средства	23
3.1 Обработка аудиодорожек	23
3.2 Смешивание аудиодорожек.....	26
3.3 Работа с графикой.....	28
3.3.1 Отрисовка рабочей области.....	28
3.3.2 Отрисовка временной шкалы	31
3.3.3 Отрисовка сетки на аудиодорожках	32
4 Тестирование программного средства	33
5 Руководство по установке и использованию программного средства	36
5.1 Взаимодействие с программным средством.....	36
5.1.1 Создание проекта.....	36
5.1.2 Импорт аудиофайлов	36
5.1.3 Создание аудиодорожек и размещение сэмплов на них	37
5.1.4 Редактирование отдельного токена	38
Заключение	40
Список использованной литературы.....	41
Приложение А. Исходный текст программы	42

ВВЕДЕНИЕ

Музыкальные редакторы становятся все более популярными в современном мире, предоставляя пользователям мощные инструменты для создания, редактирования и обработки музыкальных произведений. Эти программы позволяют как профессиональным музыкантам, так и любителям выражать свои творческие идеи, экспериментировать с различными звуковыми эффектами и создавать уникальные композиции. Музыкальные редакторы способствуют развитию таких навыков, как слух, ритм, мелодическое мышление и чувство гармонии.

С помощью музыкальных редакторов пользователи могут записывать живые инструменты, создавать электронную музыку, редактировать аудиофайлы и применять различные эффекты, такие как реверберация, эквалаизация и компрессия. Современные редакторы предлагают интуитивно понятный интерфейс, который позволяет легко манипулировать звуковыми дорожками, а также интеграцию с MIDI-устройствами для создания более сложных музыкальных произведений.

Целью курсового проекта является разработка программного средства, которое предоставит пользователю доступ к функциональному музыкальному редактору с набором инструментов для создания и редактирования музыки.

Таким образом, данный проект направлен на исследование и применение музыкальных редакторов как эффективного средства для стимулирования творческих способностей и музыкального мышления, что особенно актуально в современном обществе, где умение создавать и обрабатывать музыку становится все более востребованным навыком.

1 АНАЛИЗ ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ И ФОРМИРОВАНИЕ ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ К РАЗРАБАТЫВАЕМОМУ ПРОГРАММНОМУ СРЕДСТВУ

1.1 Обзор существующих аналогов

1.1.1 Программное средство «FL Studio»

FL Studio — это мощное и многофункциональное приложение для создания музыки, доступное на Windows и macOS. Оно предлагает интуитивный интерфейс, богатый набор инструментов и эффектов, что делает его популярным среди профессиональных музыкантов и продюсеров.

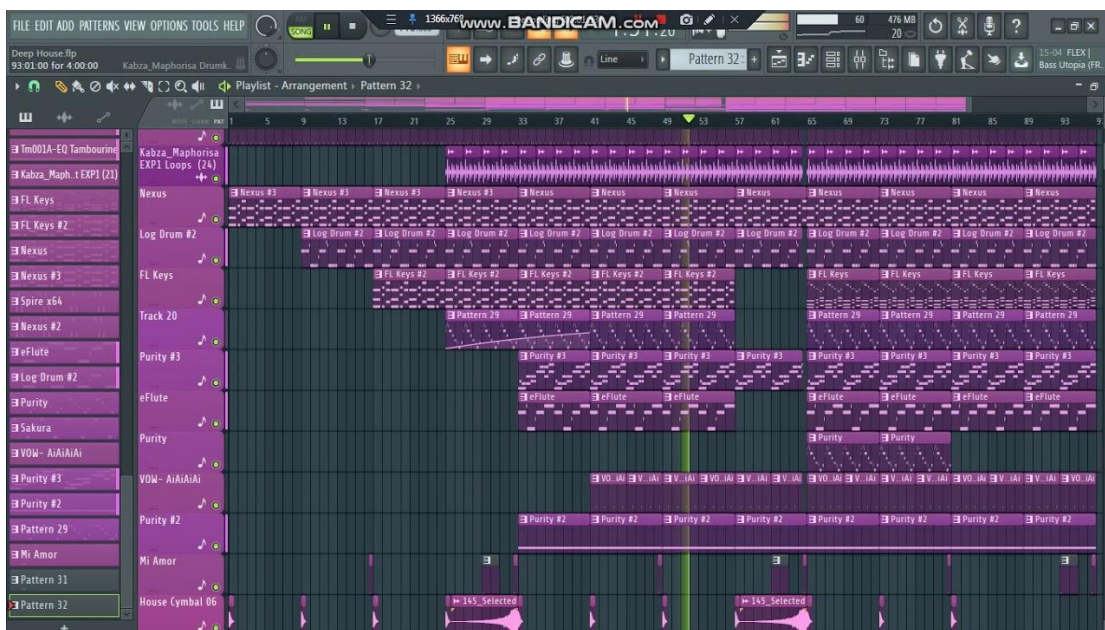


Рисунок 1.1 – главное окно «FL Studio 2020»

Преимущества:

- широкий функционал (включает секвенсоры, синтезаторы и звуковые эффекты);
- гибкость работы (позволяет создавать композиции в различных стилях и жанрах);
- поддержка плагинов (возможность расширения функциональности за счет VST и AU плагинов);
- удобный интерфейс (интуитивно понятные элементы управления и возможность настройки рабочего пространства);
- обширное сообщество (множество учебных ресурсов и форумов для обмена опытом).

Недостатки:

- высокая стоимость (премиум-версии могут быть дорогими);
- крутая кривая обучения (может быть сложным для новичков из-за

множества функций).

1.1.2 Программное средство «Audacity»

«Audacity» — это бесплатное и открытое приложение для редактирования аудио, доступное на различных платформах. Оно идеально подходит для записи и редактирования звука, что делает его популярным среди подкастеров и музыкантов.

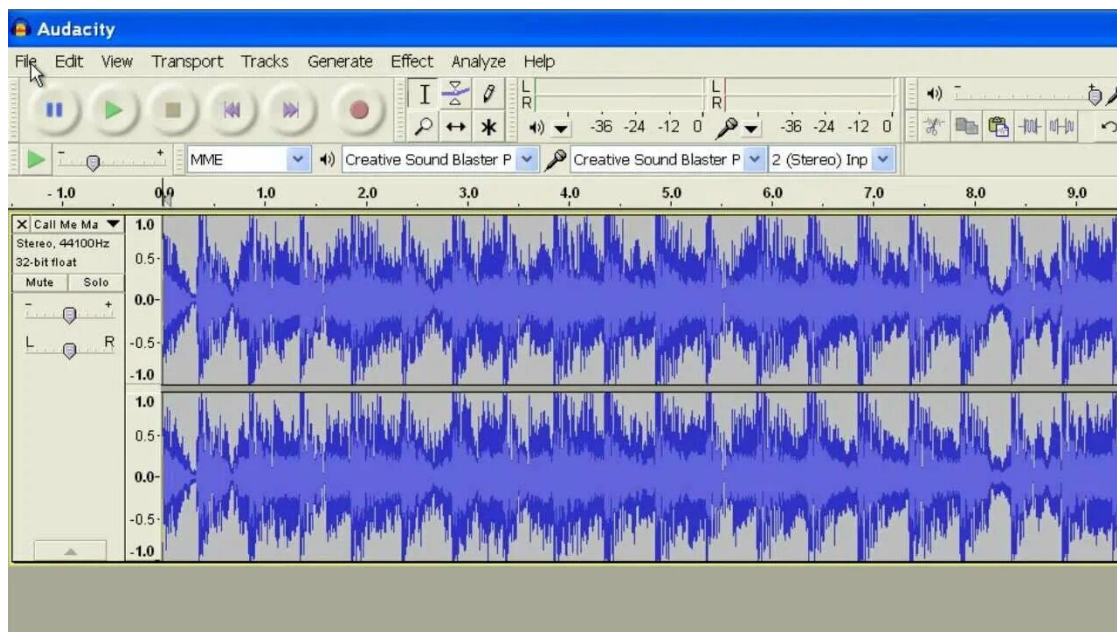


Рисунок 1.2 – главное окно программного средства «Audacity»

К преимуществам данного программного средства можно отнести:

- бесплатность (полностью бесплатное приложение без скрытых платежей);
- простота использования (интуитивно понятный интерфейс для пользователей любого уровня);
- поддержка множества форматов (возможность работы с различными аудиоформатами);
- функции редактирования (включает обрезку, смешивание и применение эффектов);
- активное сообщество (множество плагинов и учебных пособий).

К недостаткам данного программного средства можно отнести:

- ограниченные возможности для продвинутых пользователей (не хватает некоторых современных функций);
- интерфейс (может казаться устаревшим по сравнению с платными аналогами).

1.1.3 Программное средство «GarageBand»

«GarageBand» — это бесплатное приложение для создания музыки, доступное только на устройствах Apple. Оно предлагает множество инструментов и эффектов, что делает его отличным выбором для начинающих музыкантов.

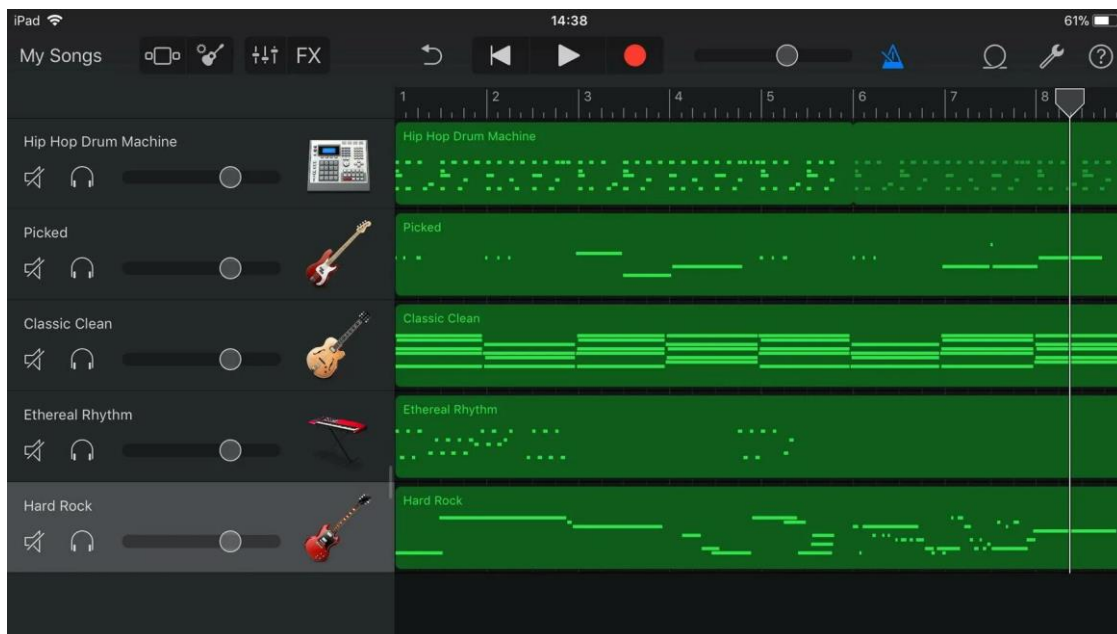


Рисунок 1.3 – главное окно программного средства «GarageBand»

Преимуществами программного средства «GarageBand» являются:

- доступность (бесплатно для пользователей macOS и iOS);
- интуитивный интерфейс (простой и понятный интерфейс, идеально подходящий для новичков);
- широкий выбор инструментов (включает виртуальные инструменты и петли);
- обучающие материалы (встроенные уроки и возможности для обучения).

Недостатки:

- ограниченная совместимость (доступно только на устройствах Apple);
- в программном средстве не хватает продвинутых функций (может не подойти для профессионального использования).

1.2 Описание средств разработки

В целях создания программного средства рассматриваются следующие подтемы:

- использование динамических структур;
- использование файлов (бинарный);

1.2.1 Использование динамических структур

Данное программное средство целесообразно разрабатывать, используя динамические структуры данных, поскольку пользователь может добавлять любое количество аудиофайлов. И для использования необходимого количества памяти в процессе создания программного средства решается задача по созданию динамической структуры данных.

Данное программное средство использует следующие структуры данных:

- связный список.

1.2.1.1 Динамическая связный список

Связный список — это структура данных, состоящая из последовательности элементов, каждый из которых содержит данные и ссылку (или указатель) на следующий элемент в последовательности. Эта структура позволяет динамически выделять память и легко добавлять или удалять элементы без необходимости перемещения других элементов, что делает связные списки более гибкими по сравнению с массивами.

Связный список будет применяться для генерации индекса случайно удаляемого элемента.

1.2.2 Использование файлов

В программе файл предстает как последовательность элементов некоторого типа. Так как размер одного файла может превышать объем всей оперативной памяти компьютера, доступ к его элементам выполняется последовательно с помощью процедур чтения напрямую из некоторого накопителя и записи непосредственно туда же.

Файлом называется какая-либо именованная последовательность байтов, взятая с диска или внешнего накопителя. Доступ к байтам данной последовательности выполняется при помощи определенных процедур чтения и записи файлов.

В C/C++ разделяются 2 различных вида файлов:

- бинарные файлы (файлы, представленные обычной последовательностью байтов и могут интерпретироваться как угодно, в зависимости от задачи);
- текстовый файл (файлы, элементы которого интерпретируются как символы).

Файлы при разработке будут применяться для сохранения данных проекта а также для хранения аудиофайлов.

1.3 Спецификация функциональных требований

Под функциональными требованиями подразумевается список задач, операций и функций, которые будут выполняться разрабатываемым программным средством.

Функциональные требования к данному программному средству указаны в таблице 1.1.

Идентификатор	Требование
ФТ-1	Проигрывания аудиотрека
ФТ-2	Сложение аудиоданных
ФТ-3	Обрезка аудиофайла
ФТ-4	Сохранение проекта
ФТ-5	Экспорт аудиофайла

Таблица 1.1– основные процедуры и функции

Комментарий к ФТ-1.

Программное средство должно проигрывать аудиодорожки. Должна быть возможность изменять позицию проигрывания.

Комментарий к ФТ-2.

Программное средство должно уметь накладывать данные аудиодорожек друг на друга, чтобы получить комбинированную аудиозапись.

Комментарий к ФТ-3.

Программное средство должно предоставлять возможность обрезки загруженных аудиофайлов.

Комментарий к ФТ-4.

Программное средство должно предоставлять возможность сохранять проект для дальнейшей работы с ним.

Комментарий к ФТ-5.

В программном средстве должна быть возможность экспортировать готовый аудиофайл после редактирования.

2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА

2.1 СТРУКТУРА ПРОГРАММЫ

При разработке программного средства будет использоваться 12 модулей:

- AudioToken – класс, который отвечает за хранение отдельных аудиотокенов на аудиодорожках.
- AudioTrack – класс, предназначенный для хранения данных аудиодорожки.
- WorkspaceModel – класс, представляющий модель для хранения списка аудиодорожек с аудиотокенами.
- AudioFileLinker – класс для хранения зависимостей между логическим аудиофайлом и физическим аудиофайлом на диске.
- AudioFileList – класс, представляющий собой список импортированных аудиофайлов.
- AudioFileObject – класс, для хранения данных о каждом загруженном аудиофайле.
- AudioTrackDialog – диалоговое окно, предназначенное для редактирования отдельного токена.
- AudioTrackFrame – класс, предназначенный для отображения аудиодорожек с токенами и предоставляющий возможности взаимодействовать с аудиоданными.
- ControlPanel – класс, предназначенный для управления программой.
- Main – класс, предназначенный для запуска QT приложения.
- MainWindow – класс для отображения виджетов главного окна.
- ProjectConfiguration – класс, предназначенный для хранения настроек проекта.

2.2 ПРОЕКТИРОВАНИЕ ИНТЕРФЕЙСА ПРОГРАММНОГО СРЕДСТВА

При разработке программного средства важным является обеспечение максимального удобства при работе с аудиоданными и их графическим отображением.

2.1.1 Главное окно

В главном окне осуществляется взаимодействие пользователя с программой. Главное окно состоит из:

- меню программы;
- списка импортированных аудиофайлов;
- панели управления;
- областью для взаимодействия с аудиоданными.

Меню программы выступает в качестве интерфейса между пользователем и основными функциями программы. В меню доступны создание, открытие и сохранение проекта, импорт аудиофайлов, экспорт отредактированного аудиофайла, а так же окно с инструкцией по эксплуатации программного средства.

Список аудиофайлов нужен для перетаскивания их на рабочую область для их обработки.

В панели управления размещены кнопки для контроля состояния проигрывания аудиодорожек.

В рабочей области будет происходить обработка данных.

Дизайн главного окна приложения представлен на рисунке 2.1.

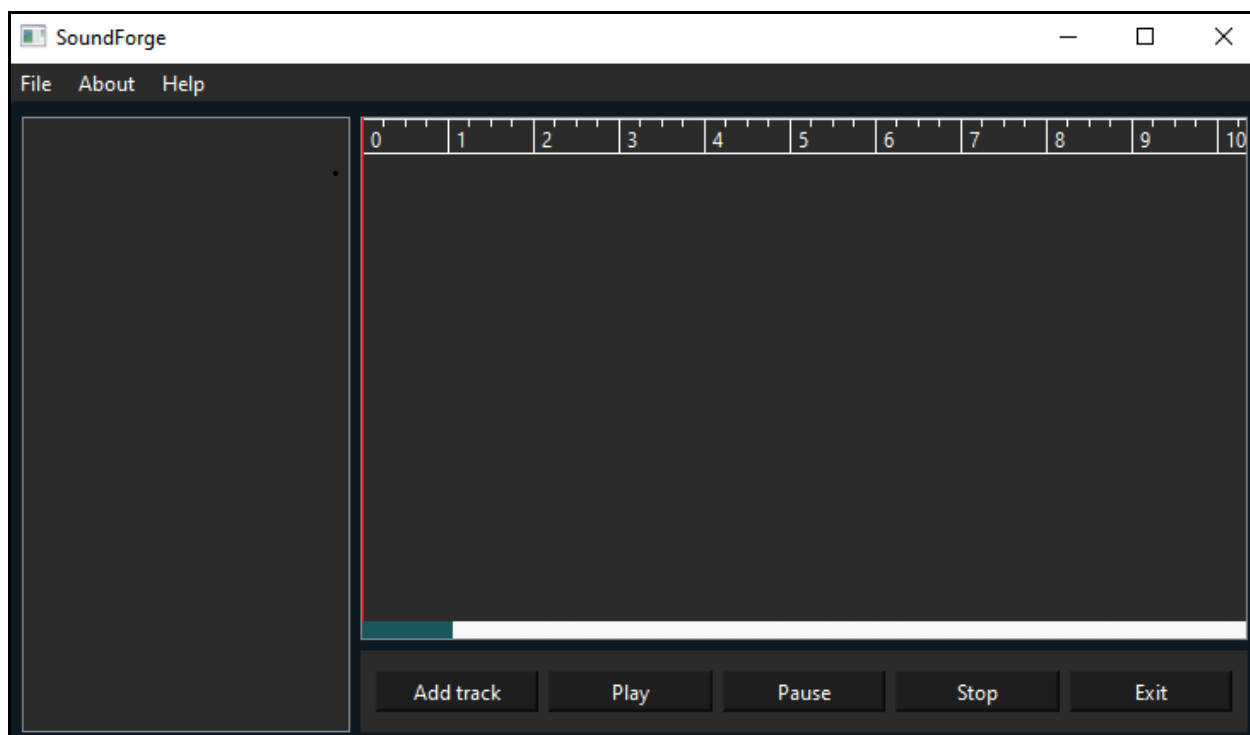


Рисунок 2.1 – главное окно приложения

2.1.2 Окно редактирования сэмпла

Окно редактирования сэмпла будет содержать в себе виджет ScrollArea внутри которого AudioTrackFrame для отображения выбранного сэмпла на аудиодорожке и предоставления инструментов для редактирования этого сэмпла.

В данном окне сэмпл можно будет разрезать с помощью инструмента “Slice”, удалить ненужные части, переставить нужные части местами.

Дизайн окна редактирования сэмпла представлен на рисунке 2.2.

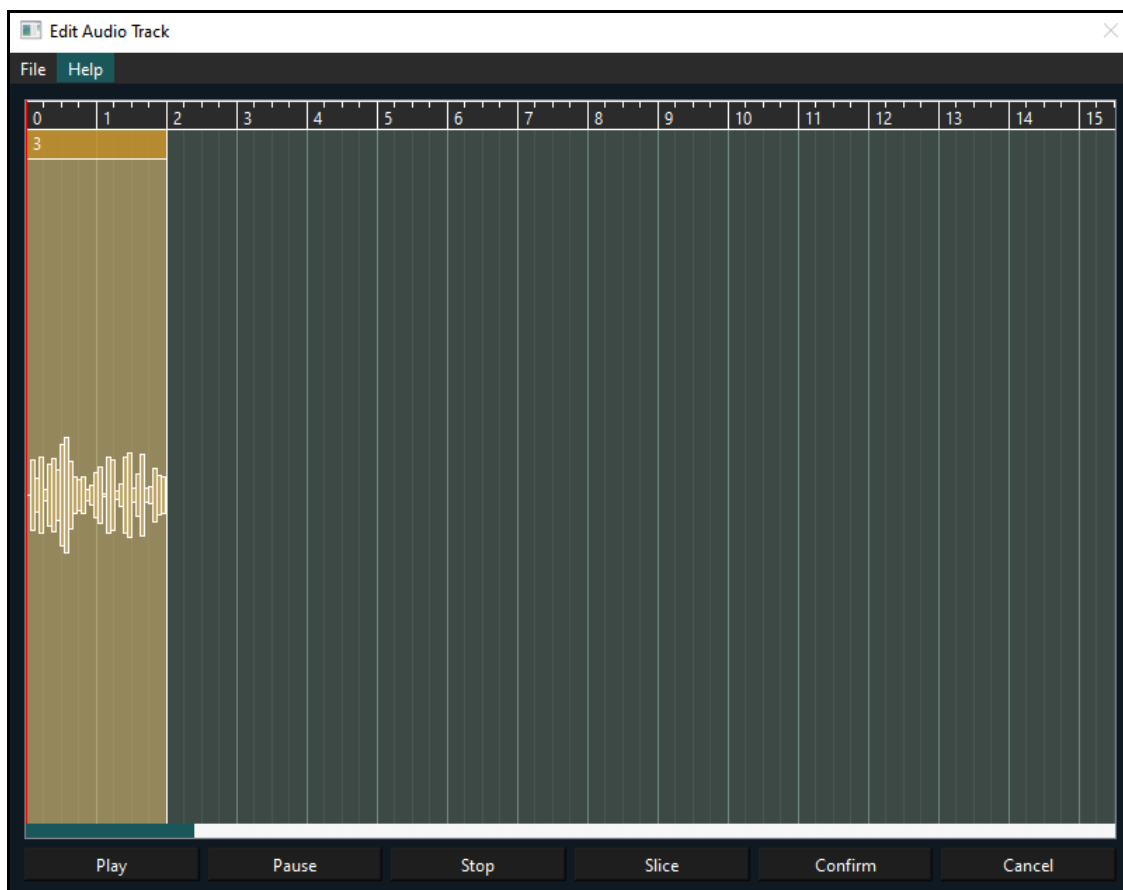


Рисунок 2.2 – окно редактирования сэмпла

2.1.3 Окно инструкции

Окно инструкции отображает краткую информацию о возможностях данного программного средства.

Дизайн данного окна представлен на рисунке 2.3.

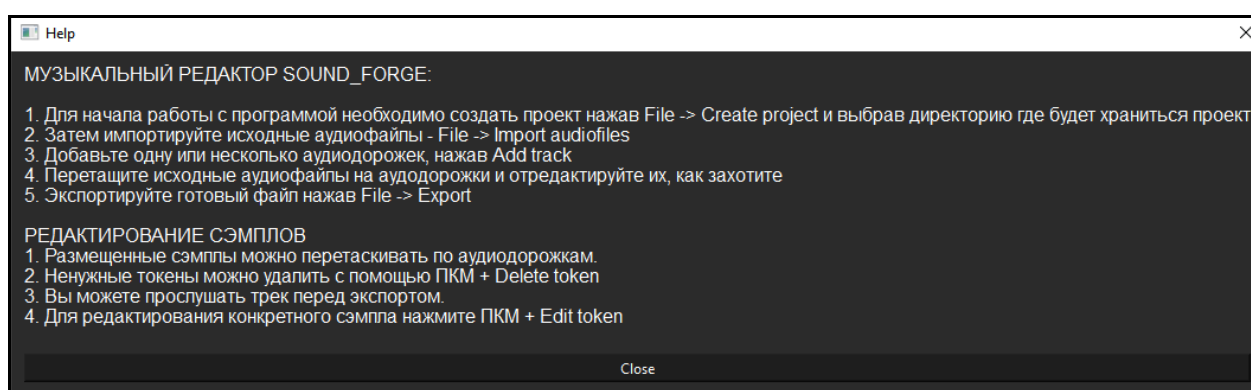


Рисунок 2.3 – окно инструкции

2.1.4 Окно о приложении

Окно о приложении будет содержать основную информацию о данном программном средстве.

Дизайн окна новой секции представлен на рисунке 2.4.

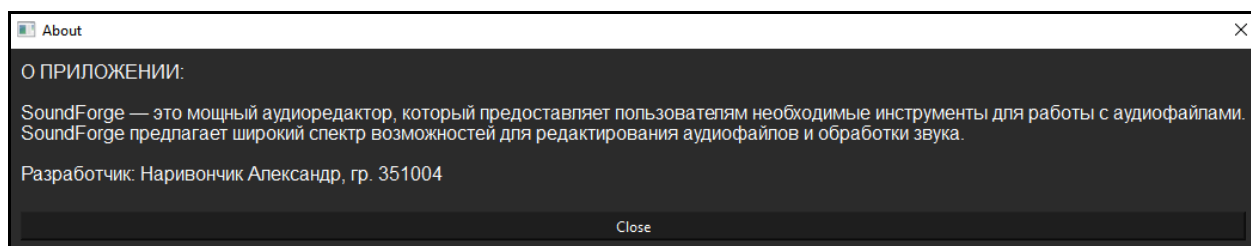


Рисунок 2.4 – окно о приложении

2.3 Проектирование функционала программного средства

Разработка алгоритмов является одной из ключевых фаз при создании программного обеспечения, особенно такого, как аудиоредактор. Для обеспечения качественного управления аудиоданными приложение должно включать в себя следующий минимальный функционал:

- проигрывания аудиотрека;
- сложение аудиоданных;
- обрезка аудиофайла;
- сохранение проекта;
- экспорт аудиофайла.

2.3.1 Проигрывание аудиотрека

Особенностью данного метода является функция проигрывания аудиотрека, которая обеспечивает плавное и эффективное воспроизведение звуковых файлов с заданной позиции. Алгоритм выполняет последовательные операции для подготовки и запуска воспроизведения трека.

Метод `playTrack` принимает параметр `startTime`, который указывает, с какой позиции следует начать воспроизведение. Сначала происходит настройка аудиовыхода с помощью метода `setAudioOutput`, после чего устанавливается уровень громкости на 80%. Если буфер уже открыт, он закрывается для избежания конфликтов с воспроизведением. Затем метод устанавливает источник устройства, используя `setSourceDevice`, и заполняет буфер данными трека.

После этого буфер открывается в режиме чтения, а аудиоплеер получает данные из буфера с помощью `setSourceDevice`. В завершение вызывается метод `play`, который запускает воспроизведение аудиотрека.

Блок-схема алгоритма данного метода приведена на рисунке 2.5.

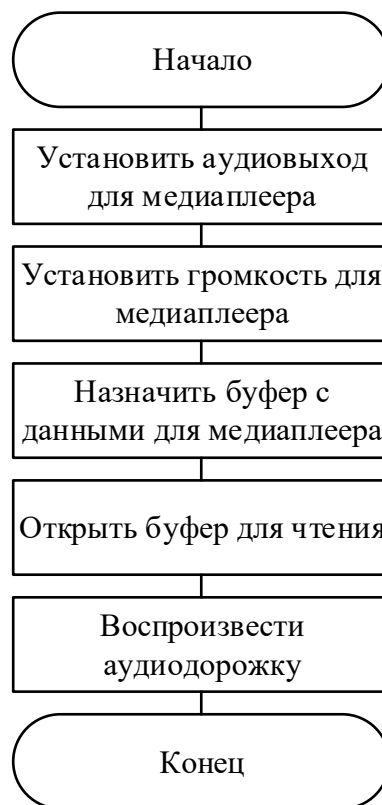


Рисунок 2.5 – блок-схема метода playTrack

Использование данного подхода позволяет эффективно управлять воспроизведением аудиофайлов, обеспечивая пользователю высокое качество звучания и возможность начать прослушивание с нужной позиции. Этот метод гарантирует, что воспроизведение трека будет настроено корректно и без задержек, создавая удобный интерфейс для работы с аудиоданными.

2.3.2 Сложение аудиоданных

Особенностью данного метода является метод смешивания двух WAV-файлов, который обеспечивает создание нового аудиотрека путем комбинирования звуковых данных. Алгоритм выполняет последовательные проверки и операции, чтобы гарантировать корректность и совместимость исходных файлов.

Метод `mixWavFiles` принимает два указателя на `QByteArray`, представляющие собой аудиофайлы. В первую очередь происходит проверка, что оба файла имеют минимальный размер заголовка в 44 байта. Если размеры меньше, выводится предупреждение, и метод возвращает пустой массив данных. Далее извлекаются заголовки обоих файлов и проверяются их параметры: количество каналов, частота дискретизации и битность. Если параметры не совпадают, также выводится предупреждение, и метод завершает работу.

Если файлы соответствуют требованиям, извлекаются параметры WAV: количество каналов, частота дискретизации и битность. Затем извлекаются аудиоданные, и определяется максимальная длина данных для смешивания. Если один из файлов короче, он дополняется нулями до длины большего файла.

Аудиоданные смешиваются путем поочередного сложения значений сэмплов из обоих файлов. Для обеспечения корректности смешивания значения ограничиваются диапазоном 16-битных целых чисел. Результирующие сэмплы записываются в новый массив.

После завершения смешивания создается новый заголовок WAV, в который обновляются размеры файла и аудиоданных. Заголовок и смешанные данные объединяются, и метод возвращает новый массив QByteArray, представляющий смешанный аудиотрек.

Блок-схема алгоритма данного метода приведена на рисунке 2.6.

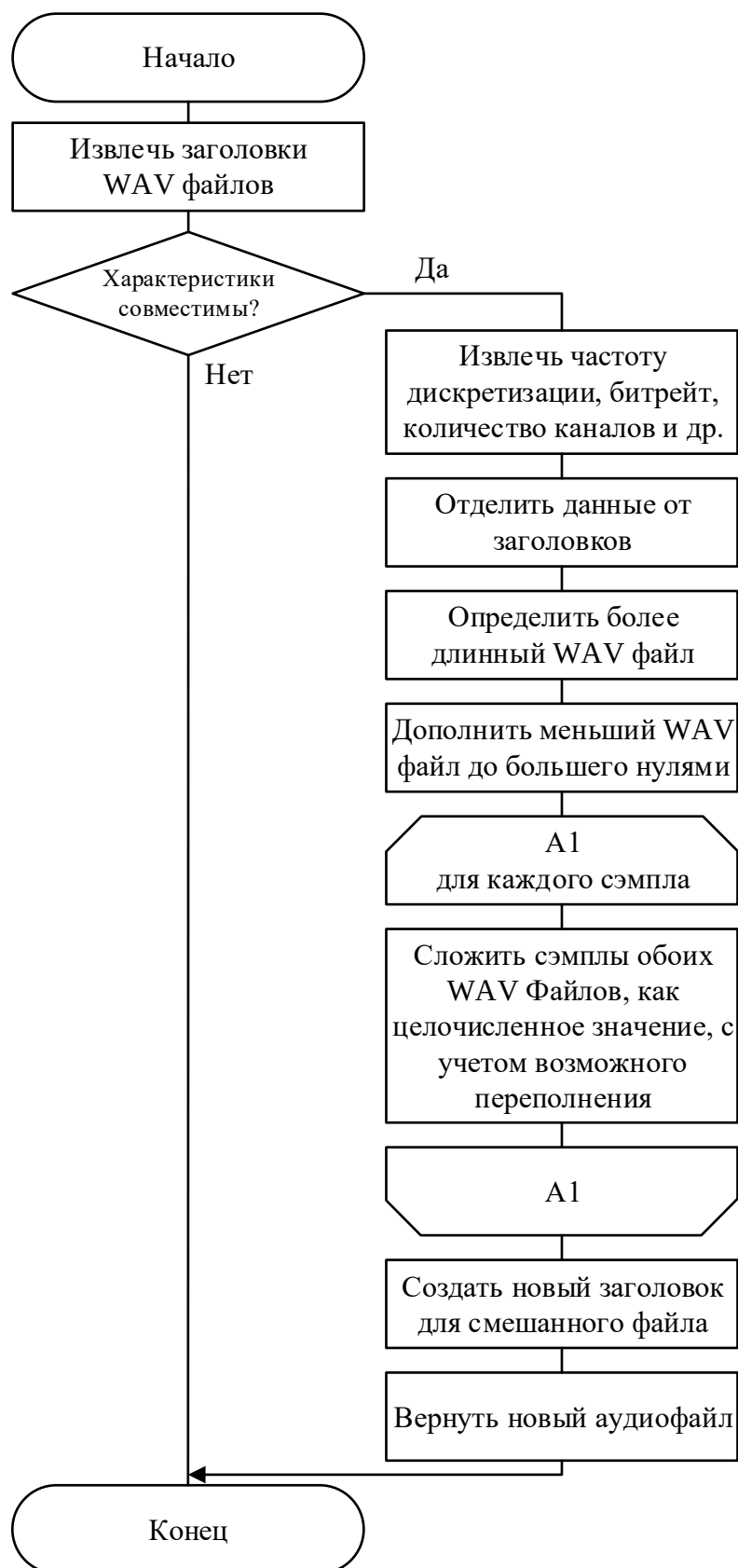


Рисунок 2.6 – блок-схема метода mixWavFiles

Использование данного подхода позволяет эффективно комбинировать аудиофайлы, создавая новые звуковые треки с высоким качеством звучания.

Этот метод обеспечивает надежность обработки данных и гибкость для пользователей, работающих с аудиофайлами.

2.3.3 Обрезка аудиофайлов

Особенностью данного метода является обработка клика левой кнопкой мыши при активном режиме разрезания аудиотреков, что обеспечивает гибкость в редактировании звуковых данных. Алгоритм выполняет проверку позиции клика и, при необходимости, разрезает выбранный токен на две части.

Метод `mouseReleaseEvent` проверяет, была ли нажата левая кнопка мыши и установлен ли флаг `isSlicing`. Если флаг активен, происходит определение позиции клика с помощью `event->pos()`. Затем вычисляется индекс дорожки на основе координаты Y клика, исключая высоту временной шкалы.

Если индекс дорожки равен нулю (только для первой дорожки), извлекается соответствующий объект `AudioTrack` из модели. Алгоритм проходит по всем токенам дорожки, извлекая информацию о каждом токене, включая его начальную позицию и длительность. Если клик попадает в границы токена, проверяется, находится ли позиция разрезания (`slicingX`) в пределах токена.

Если все условия выполнены, токен разрезается на два новых. Первый новый токен сохраняет часть оригинального токена, а второй начинается с позиции разрезания и продолжает оставшуюся часть. Исходный токен удаляется из списка, и вместо него добавляются два новых токена.

После успешного разрезания сбрасывается флаг `isSlicing`, чтобы предотвратить дальнейшие операции разрезания, и интерфейс обновляется методом `update()`, что позволяет пользователю увидеть изменения. Блок-схема алгоритма метода `sliceToken` представлена на рисунке 2.7.

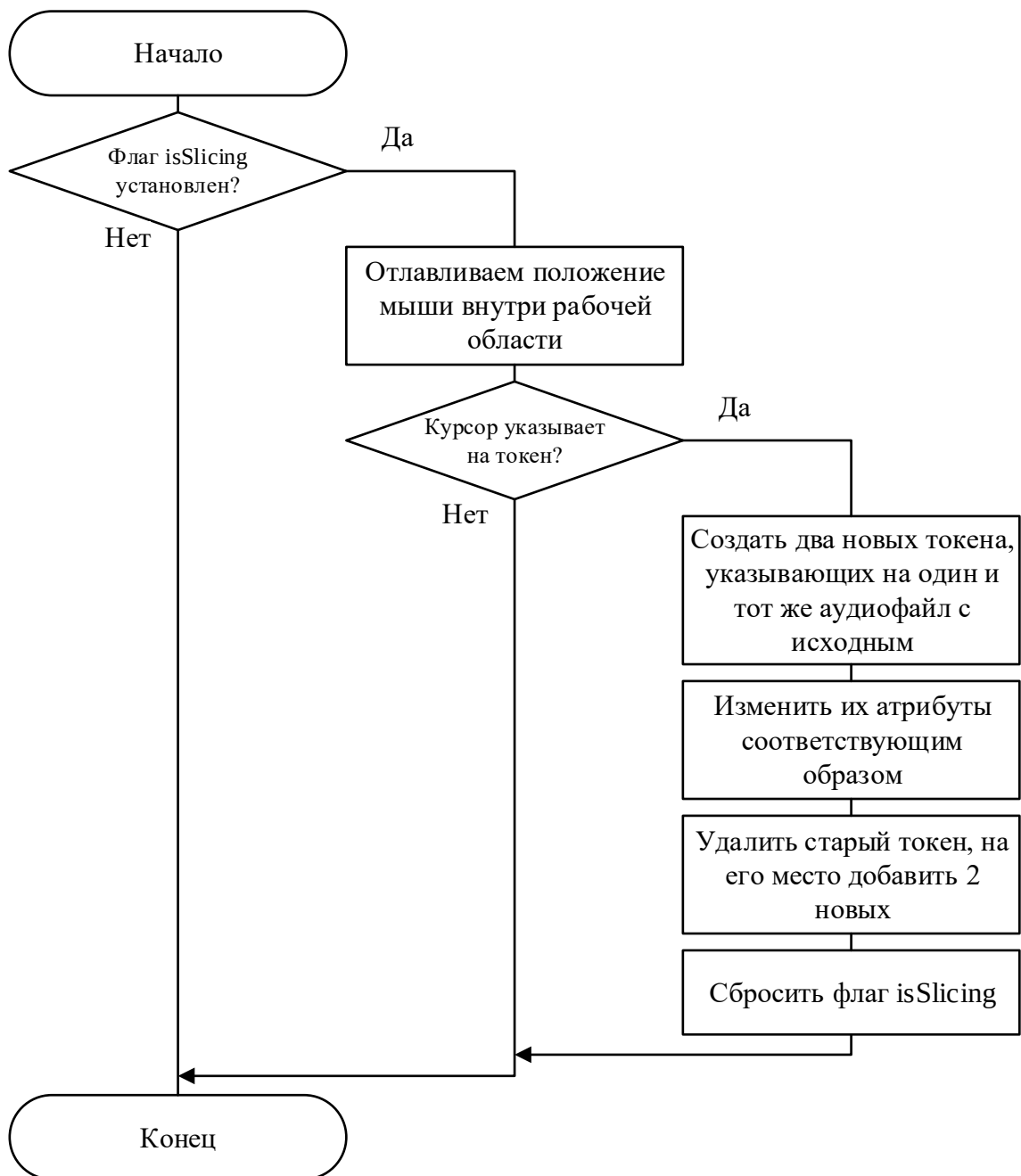


Рисунок 2.7 – Блок-схема метода sliceToken

Использование данного подхода позволяет пользователям легко разделять аудиофайлы и обрезать их, обеспечивая удобство и точность в процесс.

2.3.4 Сохранение проекта

Особенностью данного метода является процесс сохранения проекта, который обеспечивает запись всех аудиотреков в файл. Алгоритм выполняет последовательные проверки и операции, чтобы гарантировать корректное сохранение информации.

Сначала создается объект QFile, который инициализируется полным путем к проекту. Если файл не удастся открыть для записи, также выводится предупреждение, и метод завершает свою работу.

Для каждой дорожки происходит проход по всем токенам, которые она содержит. Для каждого токена в файл записываются его атрибуты: идентификатор аудиофайла, начальная позиция, длительность, относительное время начала и относительная длительность, а также индекс дорожки. Эти данные записываются в файл поочередно, что обеспечивает полное сохранение информации о каждом токене.

Когда все данные записаны, файл закрывается явным вызовом метода close(), что завершает процесс сохранения. В конце выводится информационное сообщение о том, что проект был успешно сохранен.

Блок-схема алгоритма метода saveProject представлена на рисунке 2.8.

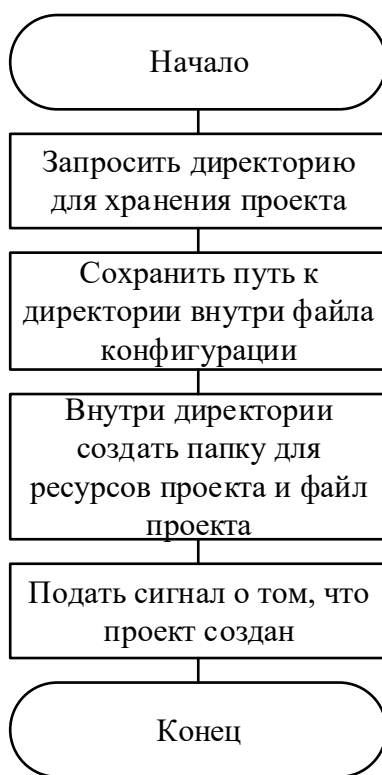


Рисунок 2.8 – Блок-схема метода saveProject

2.3.5 Экспорт аудиофайла

Данный метод предназначен для экспорта аудиофайла из проекта и позволяет пользователю сохранить смешанный аудиотрек в формате WAV. Алгоритм выполняет последовательную обработку и запись аудиоданных в выбранный пользователем файл.

В начале работы пользователю предлагается выбрать путь для сохранения файла с помощью стандартного диалогового окна Windows, вызываемого через функцию GetSaveFileNameW. Если пользователь выбрал

путь и подтвердил сохранение, метод получает итоговый путь к файлу и переходит к следующему этапу.

После получения корректного пути выполняется создание объекта QFile, и проверяется возможность открытия выбранного файла для записи. Если файл успешно открыт, начинается обработка и подготовка аудиодорожек проекта. Далее вызывается метод `mixWavFiles()`, который смешивает аудиоданные основной дорожки с остальными дорожками. Результат смешивания сохраняется в `trackData` основной аудиодорожки.

Завершающим этапом является запись полученных смешанных аудиоданных в WAV-файл по выбранному пути. Блок-схема алгоритма метода `exportAudiofile` представлена на рисунке 2.9.

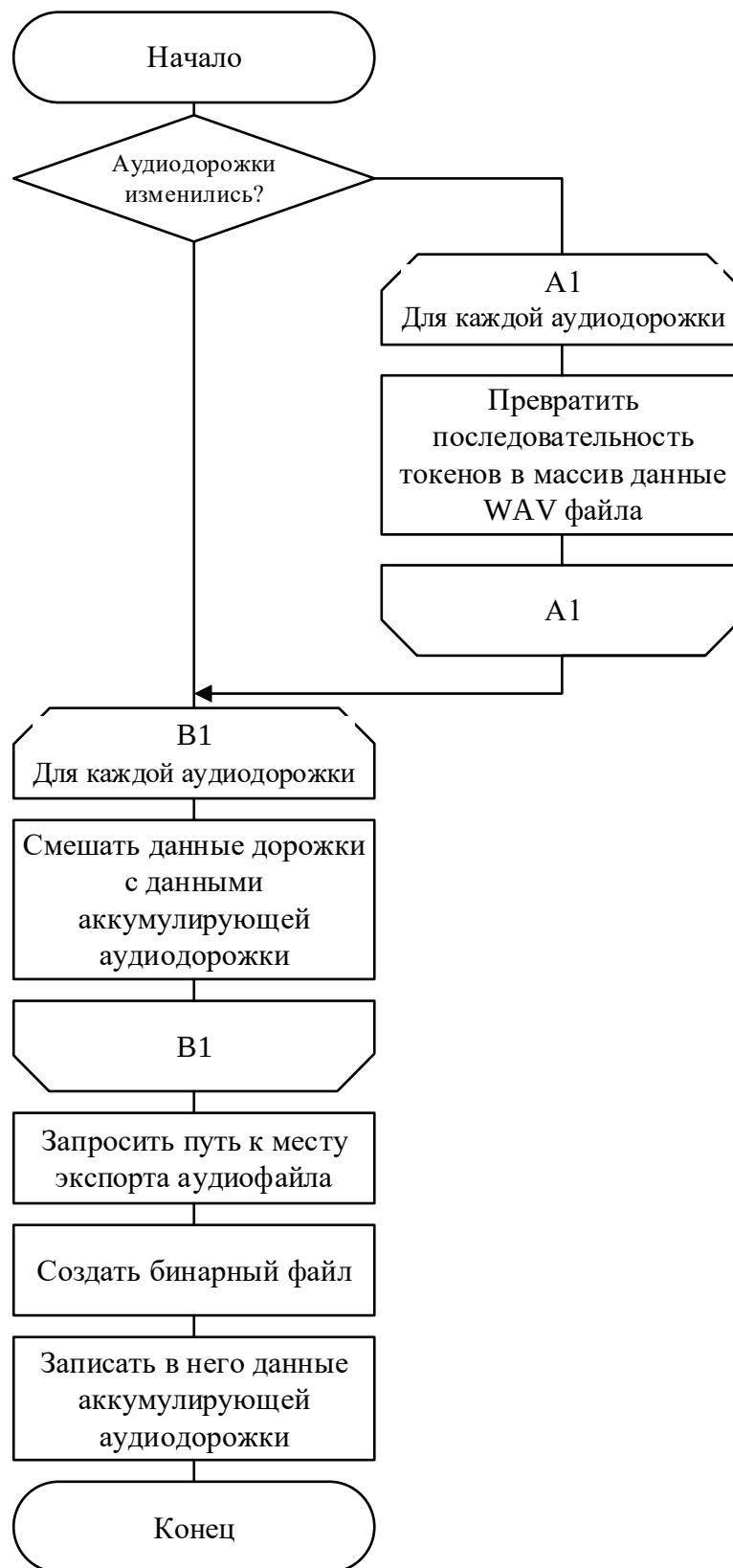


Рисунок 2.9 – Блок-схема метода exportAudiofile

3 РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА

3.1 Обработка аудиодорожек

Алгоритм обработки аудиодорожки начинается с создания заголовка WAV, для чего используется функция `createWavHeader`. Эта функция формирует заголовок, который содержит ключевую информацию о формате аудио, включая метаданные, такие как количество каналов, частота дискретизации и размер данных. Заголовок создается с нулевым размером данных, так как фактический размер будет определен позже, после обработки всех токенов.

```
QByteArray createWavHeader(int dataSize) {
    // Создаем заголовок WAV размером WAV_HEADER_SIZE и заполняем его нулями
    QByteArray header(WAV_HEADER_SIZE, 0);
    char *headerData = header.data(); // Получаем указатель на данные заголовка

    // RIFF заголовок
    std::memcpy(headerData, "RIFF", 4); // Записываем строку "RIFF" в заголовок
    int chunkSize = dataSize + 36; // Общий размер файла - 8 байт (RIFF заголовок и
    WAVE)
    std::memcpy(headerData + 4, &chunkSize, 4); // Записываем размер файла в
    заголовок
    std::memcpy(headerData + 8, "WAVE", 4); // Записываем строку "WAVE" в заголовок

    // fmt подзаголовок
    std::memcpy(headerData + 12, "fmt ", 4); // Записываем "fmt " в заголовок
    int subchunk1Size = 16; // Размер подзаголовка (16 для PCM)
    std::memcpy(headerData + 16, &subchunk1Size, 4); // Записываем размер
    подзаголовка
    short audioFormat = 1; // PCM формат
    std::memcpy(headerData + 20, &audioFormat, 2); // Записываем формат аудио
    std::memcpy(headerData + 22, &CHANNELS, 2); // Записываем количество каналов
    std::memcpy(headerData + 24, &SAMPLE_RATE, 4); // Записываем частоту
    дискретизации
    int byteRate = SAMPLE_RATE * BLOCK_ALIGN; // Рассчитываем скорость передачи
    байтов
    std::memcpy(headerData + 28, &byteRate, 4); // Записываем скорость передачи
    байтов
    std::memcpy(headerData + 32, &BLOCK_ALIGN, 2); // Записываем размер блока
    std::memcpy(headerData + 34, &BITS_PER_SAMPLE, 2); // Записываем битность

    // data подзаголовок
    std::memcpy(headerData + 36, "data", 4); // Записываем "data" в заголовок
    std::memcpy(headerData + 40, &dataSize, 4); // Записываем размер данных

    return header; // Возвращаем сформированный заголовок
}
```

Следующий шаг — это сортировка токенов. В методе `processAudioTrack` токены сортируются по возрастанию их начальных позиций с помощью стандартной функции `std::sort`. Это необходимо для того, чтобы гарантировать правильный порядок аудиоданных в результирующем массиве, так как токены могут быть добавлены в произвольном порядке.

После сортировки начинается инициализация результирующего массива `combinedAudioData`, который предназначен для хранения всех объединенных аудиоданных. Изначально этот массив заполняется заголовком WAV, который будет обновлен позже, когда известен фактический размер данных. Переменная `currentBytePos` используется для отслеживания текущей позиции в массиве в байтах.

Далее следует цикл, который проходит по всем токенам в отсортированном списке. Для каждого токена алгоритм извлекает соответствующие аудиоданные из файловой карты, пропуская заголовок, чтобы работать только с аудиоформатом. Затем рассчитываются начальная позиция и длина активной части данных токена в байтах.

Если между текущей позицией в результирующем массиве и началом токена есть промежуток, этот промежуток заполняется тишиной. Для этого вычисляется количество байтов, необходимых для тишины, и добавляется в массив в виде нулей. После этого проверяется, кратен ли размер данных токена заданному значению `BLOCK_ALIGN`. Если нет, добавляются дополнительные нули, чтобы достичь нужной кратности.

Затем извлеченные данные токена добавляются в общий массив, и текущая позиция обновляется. Этот процесс повторяется для всех токенов в списке, обеспечивая корректное размещение всех аудиоданных в результирующем массиве.

Когда все токены обработаны, алгоритм обновляет размер данных в заголовке WAV. Для этого вызывается функция `createWavHeader` с актуальным размером данных, и новый заголовок записывается в начало массива. В конце метод сохраняет итоговый массив с объединенными аудиоданными в поле `trackData` и устанавливает флаг `isTrackChanged` в значение `false`, что указывает на отсутствие изменений в треке после завершения обработки.

Таким образом, алгоритм обеспечивает последовательную обработку и объединение аудиоданных из различных токенов, формируя готовый аудиотрек в формате WAV. Код метода обработки аудиодорожки представлен ниже.

```
void AudioTrack::processAudioTrack() {
    // 1. Сортировка токенов по возрастанию startPositionView
    std::sort(tokens.begin(), tokens.end(),
        [](const AudioToken &a, const AudioToken &b) {
            return a.startPositionView < b.startPositionView; // Сравниваем
позиции начала токенов
        });

    // 2. Объявляем результирующий массив для данных
    QByteArray *combinedAudioData = new QByteArray();

    // Изначально заполняем массив заголовком WAV
    combinedAudioData->append(createWavHeader(0)); // Заголовок обновим позже
```



```

// Переменная для отслеживания текущей позиции в байтах
int currentBytePos = 0;

// Проходим по всем токенам
for (const AudioToken &token : tokens) {
    // Получаем аудиоданные из карты файлов
    QByteArray audioFileData = AudioFileLinker::audioFiles[token.audiofileID]-
>audioData;

    QByteArray audioData = audioFileData.mid(WAV_HEADER_SIZE); // Извлекаем
данные без заголовка

    // Рассчитываем смещение и длину активной части в байтах
    int startByte = static_cast<int>(token.relativeStartTimeView * MS_TO_PX *
MS_TO_BYTES);
    int durationBytes = static_cast<int>(token.relativeDurationView * MS_TO_PX *
MS_TO_BYTES);

    // Извлекаем активную часть аудиоданных токена
    QByteArray tokenAudioData = audioData.mid(startByte, durationBytes);

    // Рассчитываем позицию начала токена в результирующем массиве
    int tokenStartPos = static_cast<int>(token.startPositionView * MS_TO_PX *
MS_TO_BYTES);

    // Если между текущей позицией и началом токена есть промежуток, заполняем
его тишиной
    if (currentBytePos < tokenStartPos) {
        int silenceBytes = tokenStartPos - currentBytePos; // Вычисляем
количество байтов тишины
        silenceBytes += (BLOCK_ALIGN - (silenceBytes % BLOCK_ALIGN)) %
BLOCK_ALIGN; // Дополняем до кратного BLOCK_ALIGN
        combinedAudioData->append(QByteArray(silenceBytes, 0)); // Добавляем
тишину в массив
        currentBytePos = tokenStartPos; // Обновляем текущую позицию
    }

    // Проверяем кратность размера данных BLOCK_ALIGN
    if (tokenAudioData.size() % BLOCK_ALIGN != 0) {
        int padding = BLOCK_ALIGN - (tokenAudioData.size() % BLOCK_ALIGN); //
Вычисляем недостающие байты
        tokenAudioData.append(QByteArray(padding, 0)); // Дополняем до кратного
BLOCK_ALIGN
    }

    // Добавляем данные токена в общий массив
    combinedAudioData->append(tokenAudioData);
    currentBytePos += tokenAudioData.size(); // Обновляем текущую позицию
}

// Обновляем размер data в заголовке WAV
int dataSize = combinedAudioData->size() - WAV_HEADER_SIZE; // Вычисляем размер
данных
QByteArray header = createWavHeader(dataSize); // Создаем новый заголовок с
актуальным размером
std::memcpy(combinedAudioData->data(), header.data(), WAV_HEADER_SIZE); //
Обновляем заголовок в массиве

trackData = combinedAudioData; // Сохраняем результирующий массив в trackData
isTrackChanged = false; // Сбрасываем флаг изменения трека
}

```

Таким образом, алгоритм обеспечивает последовательную обработку и объединение аудиоданных из различных токенов, формируя готовый аудиотрек в формате WAV.

3.2 Смешивание аудиодорожек

Метод `mixWavFiles` отвечает за смешивание двух WAV-файлов, и его работа начинается с проверки, что оба файла имеют достаточный размер, чтобы включать хотя бы заголовок. Если размер любого из файлов меньше 44 байт, что является минимальным размером заголовка WAV, выводится предупреждение, и метод завершает выполнение, возвращая пустой массив `QByteArray`.

Затем из обоих файлов извлекаются заголовки. Заголовки представляют собой первые 44 байта каждого из файлов, что позволяет впоследствии проверить, совпадают ли параметры аудио. Метод проверяет, что количество каналов, частота дискретизации и битность (разрядность) аудиоданных совпадают для обоих файлов. Если параметры различаются, выводится предупреждение о несовпадении форматов, и метод также завершает выполнение с пустым результатом.

После успешной проверки параметров алгоритм извлекает конкретные значения, такие как количество каналов, частота дискретизации и битность, а также рассчитывает количество байтов на один сэмпл. Важно отметить, что метод поддерживает только 16-битные WAV-файлы; если битность отличается, выводится предупреждение, и метод завершает выполнение.

Далее извлекаются аудиоданные из обоих файлов, начиная с 45 байта, так как первые 44 байта занимают заголовок. Затем определяется максимальная длина аудиоданных, что позволяет дополнить меньший файл нулями до длины большего файла. Это необходимо для корректного смешивания, чтобы избежать выхода за пределы массива.

После подготовки данных начинается процесс смешивания аудиоданных. Создается новый массив `mixedData`, который будет хранить результат смешивания и который инициализируется размером максимальной длины. Алгоритм проходит по всем сэмплам в цикле, извлекая 16-битные значения из обоих массивов и суммируя их. Полученное значение смешанного сэмпла ограничивается диапазоном от `INT16_MIN` до `INT16_MAX`, чтобы избежать искажения из-за переполнения.

Каждый смешанный сэмпл записывается в результирующий массив `mixedData`. После того как все сэмплы обработаны, создается новый заголовок WAV, который начинается с заголовка первого файла. Обновляются размеры файла и аудиоданных в заголовке на основе размера смешанных данных, что позволяет правильно формировать итоговый файл.

Код описанного метода представлен ниже.

```
QByteArray AudioTrackFrame::mixWavFiles(QByteArray *wav1, QByteArray *wav2) {  
    // Проверяем, что оба файла имеют минимум заголовок (44 байта)
```

```

    if (wav1->size() < 44 || wav2->size() < 44) {
        qWarning() << "Invalid WAV files: too small!"; // Вывод предупреждения о
        некорректных файлах
        return QByteArray(); // Возвращаем пустой массив, если файлы слишком
        маленькие
    }

    // Извлекаем заголовки из обоих WAV-файлов
    QByteArray header1 = wav1->left(44); // Заголовок первого файла
    QByteArray header2 = wav2->left(44); // Заголовок второго файла

    // Проверяем, что параметры WAV-файлов совпадают
    if (header1.mid(22, 2) != header2.mid(22, 2) || // Сравниваем количество каналов
        header1.mid(24, 4) != header2.mid(24, 4) || // Сравниваем частоту
        дискретизации
        header1.mid(34, 2) != header2.mid(34, 2)) { // Сравниваем битность
        qWarning() << "WAV files have different formats!"; // Вывод предупреждения о
        несовпадении форматов
        return QByteArray(); // Возвращаем пустой массив, если форматы различаются
    }

    // Извлекаем параметры WAV
    int16_t numChannels = qFromLittleEndian<int16_t>(reinterpret_cast<const uchar
*>(header1.data() + 22)); // Количество каналов
    int32_t sampleRate = qFromLittleEndian<int32_t>(reinterpret_cast<const uchar
*>(header1.data() + 24)); // Частота дискретизации
    int16_t bitsPerSample = qFromLittleEndian<int16_t>(reinterpret_cast<const uchar
*>(header1.data() + 34)); // Битность
    int16_t bytesPerSample = bitsPerSample / 8; // Количество байтов на сэмпл

    // Проверяем, что битность равна 16
    if (bitsPerSample != 16) {
        qWarning() << "Only 16-bit WAV files are supported!"; // Вывод предупреждения
        о неподдерживаемом формате
        return QByteArray(); // Возвращаем пустой массив, если битность не 16
    }

    // Извлекаем аудиоданные из файлов, пропуская заголовки
    QByteArray data1 = wav1->mid(44); // Аудиоданные первого файла
    QByteArray data2 = wav2->mid(44); // Аудиоданные второго файла

    // Определяем длину аудиоданных
    int maxLength = qMax(data1.size(), data2.size()); // Длина большего из двух
    массивов

    // Дополняем меньший файл нулями до длины большего
    if (data1.size() < maxLength) {
        data1.append(QByteArray(maxLength - data1.size(), 0)); // Добавляем нули к
        первому файлу
    }
    if (data2.size() < maxLength) {
        data2.append(QByteArray(maxLength - data2.size(), 0)); // Добавляем нули ко
        второму файлу
    }

    // Смешиваем аудиоданные
    QByteArray mixedData; // Массив для смешанных данных
    mixedData.resize(maxLength); // Устанавливаем размер массива равным максимальной
    длине

    // Проходим по каждому сэмплу

```

```

        for (int i = 0; i < maxLength; i += bytesPerSample) {
            // Берем сэмплы как 16-битные значения
            int16_t sample1 = qFromLittleEndian<int16_t>(reinterpret_cast<const uchar
*>(data1.data() + i)); // Первый сэмпл
            int16_t sample2 = qFromLittleEndian<int16_t>(reinterpret_cast<const uchar
*>(data2.data() + i)); // Второй сэмпл

            // Смешиваем сэмплы
            int32_t mixedSample = sample1 + sample2; // Суммируем сэмплы

            // Ограничиваем диапазон значения
            if (mixedSample > INT16_MAX) mixedSample = INT16_MAX; // Ограничиваем верхний
предел
            if (mixedSample < INT16_MIN) mixedSample = INT16_MIN; // Ограничиваем нижний
предел

            // Добавляем смешанный сэмпл в результирующий массив
            qToLittleEndian<int16_t>(static_cast<int16_t>(mixedSample),
reinterpret_cast<uchar *>(mixedData.data() + i)); // Записываем смешанный сэмпл
        }

        // Создаем новый заголовок WAV
        QByteArray newHeader = header1; // Используем заголовок первого файла
        int32_t newDataSize = mixedData.size(); // Размер новых данных
        int32_t newFileSize = 36 + newDataSize; // Общий размер файла

        // Обновляем размер файла в заголовке
        qToLittleEndian<int32_t>(newFileSize, reinterpret_cast<uchar *>(newHeader.data()
+ 4)); // Записываем новый размер файла в заголовок
        // Обновляем размер аудиоданных в заголовке
        qToLittleEndian<int32_t>(newDataSize, reinterpret_cast<uchar *>(newHeader.data()
+ 40)); // Записываем новый размер данных в заголовок

        // Соединяем заголовок и данные
        QByteArray result = newHeader + mixedData; // Объединяем заголовок и смешанные
данные
        return result; // Возвращаем результат
    }

```

Наконец, заголовок и смешанные данные объединяются в один массив `result`, который возвращается как результат работы метода. Таким образом, метод `mixWavFiles` обеспечивает корректное смешивание двух WAV-файлов с учетом всех необходимых проверок и преобразований, формируя новый WAV-файл, содержащий объединенные аудиоданные.

3.3 Работа с графикой

За отрисовку графического интерфейса отвечает метод `paintEvent` класса `AudioTrackFrame`. Он использует класс `QPainter` для создания визуальных элементов и отображения информации о треках, токенах и временной шкале.

3.3.1 Отрисовка рабочей области

Начинается метод `paintEvent` с вызова метода отрисовки родительского класса `QFrame`, что обеспечивает стандартную отрисовку. Затем проверяется,

существует ли модель данных (model). Если модель отсутствует, метод просто завершает выполнение без дальнейших действий.

После этого создается объект QPainter, который будет использоваться для рисования на текущем виджете. Рассчитывается ширина трека, исходя из количества тактов, их длительности и коэффициента масштаба. Это позволяет адаптировать размеры элементов в зависимости от текущего состояния интерфейса.

Затем начинается отрисовка дорожек. В цикле по всем дорожкам модели устанавливается цвет фона для каждой дорожки. Если дорожка выбрана, используется один цвет, в противном случае — другой. Рисуются прямоугольник, представляющий дорожку, с правильными размерами и положением.

После этого вызывается метод drawTrackGrid, который рисует сетку на дорожках. Эта сетка помогает визуально разделить дорожки на такты и четверти, что упрощает восприятие структуры аудиотрека.

Затем переходим к отрисовке токенов, которые представляют аудиоданные на дорожках. Если токен перетаскивается, сначала отрисовываются все токены, кроме того, который в данный момент перетаскивается. Если токен не перетаскивается, то рисуются все токены на дорожке. Это позволяет пользователю видеть все токены, кроме того, который он перемещает.

Если токен перетаскивается, рисуется его верхняя и нижняя части с установленными цветами. Позиции и размеры частей токена рассчитываются в зависимости от его текущего положения и высоты дорожки, что делает интерфейс более интуитивно понятным.

Далее вызывается метод drawTimeBar, который рисует временную шкалу в верхней части интерфейса. Это позволяет пользователю видеть, где именно на временной шкале находится аудиотрек.

После этого метод рисует линию, которая указывает текущую позицию воспроизведения. Эта линия красного цвета помогает пользователю быстро ориентироваться, где в данный момент находится прогресс воспроизведения аудиотрека.

Кроме того, если в данный момент происходит нарезка, рисуется линия нарезки зеленого цвета, чтобы визуально обозначить это действие. В конце метода вызывается update, чтобы обновить виджет, если требуется.

Код отрисовки основной области представлен ниже.

```
void AudioTrackFrame::paintEvent(QPaintEvent *event) {
    QFrame::paintEvent(event); // Вызываем отрисовку родительского класса
    if (!model) return; // Проверяем, существует ли модель (данные треков)

    QPainter painter(this); // Создаем объект QPainter, который будет рисовать на
    // текущем виджете
    int trackWidth = trackTactCount * tactDuration * scaleFactor; // Масштабируем
    // ширину трека

    // Рисуем дорожки
```

```

for (int i = 0; i < model->rowCount(); i++) {
    AudioTrack *track = model->tracks[i]; // Получаем текущий трек из модели

    // Устанавливаем цвет фона дорожки в зависимости от выбора
    painter.setBrush((track->isSelected) ?
ProjectConfiguration::clSelectedAudioTrack : ProjectConfiguration::clAudioTrack);
    // Рисуем прямоугольник, представляющий дорожку
    painter.drawRect(0, TIME_BAR_HEIGHT + i * trackHeight, trackWidth,
trackHeight);
}

// Рисуем сетку для дорожек (каждые 1/4 такта)
drawTrackGrid(painter, model->rowCount());

// Рисуем токены на дорожке
for (int i = 0; i < model->rowCount(); i++) {
    AudioTrack *track = model->tracks[i]; // Получаем текущий трек

    // Проверяем, перетаскиваем ли токен
    if (isTokenDragging && draggedToken->audioTrack == i) {
        // Если да, рисуем все токены, кроме перетаскиваемого
        for (int j = 0; j < track->tokens.size(); j++) {
            if (draggedTokenIndex != j) {
                AudioToken token = track->tokens.at(j);
                token.drawToken(&painter, scaleFactor, trackHeight); // Рисуем
токен с учетом масштаба
            }
        }
    } else {
        // Если токен не перетаскивается, рисуем все токены на дорожке
        for (AudioToken token : track->tokens) {
            token.drawToken(&painter, scaleFactor, trackHeight); // Рисуем токен
с учетом масштаба
        }
    }

    // Рисуем перетаскиваемый токен
    if (isTokenDragging) {
        painter.setBrush(Qt::red); // Устанавливаем цвет для перетаскиваемого
токена
        int x = draggedTokenStartX; // Получаем начальную позицию по X
        int targetTrack = (draggedTokenStartY) / trackHeight; // Определяем
целевую дорожку по Y
        if (targetTrack >= 0 && targetTrack < model->rowCount() && x >= 0) {
            int headerHeight = TRACK_HEIGHT * TOKEN_HEADER_RELATIVE_HEIGHT; //
Высота заголовка токена
            int mainContentHeight = trackHeight - headerHeight; // Высота
основного содержимого токена

            // Верхняя часть токена с текстом
            painter.setPen(ProjectConfiguration::clTokenText); // Устанавливаем
цвет текста
            painter.setBrush(DARK_DRAGGED_TOKEN_COLOR); // Устанавливаем цвет
верхней части токена
            painter.drawRect(x, TIME_BAR_HEIGHT + targetTrack * trackHeight,
draggedToken->relativeDurationView * scaleFactor, headerHeight); // Рисуем верхнюю
часть токена

            // Нижняя часть для отображения аудиосигнала
            painter.setBrush(LIGHT_DRAGGED_TOKEN_COLOR); // Устанавливаем цвет
нижней части токена

```

```

        painter.drawRect(x, TIME_BAR_HEIGHT + targetTrack * trackHeight +
headerHeight, draggedToken->relativeDurationView * scaleFactor, mainContentHeight);
// Рисуем нижнюю часть токена
    }
}

// Рисуем временную шкалу
drawTimeBar(painter, trackWidth);

// Рисуем текущую позицию проигрывания
painter.setPen(Qt::red); // Устанавливаем цвет линии текущей позиции
painter.drawLine(currViewTime * scaleFactor, 0, currViewTime * scaleFactor,
height()); // Рисуем вертикальную линию текущей позиции с учетом масштаба

// Если происходит нарезка, рисуем линию нарезки
if (isSlicing) {
    painter.setPen(Qt::green); // Устанавливаем цвет линии нарезки
    painter.drawLine(slicingX * scaleFactor, 0, slicingX * scaleFactor,
height()); // Рисуем вертикальную линию нарезки
    update(); // Обновляем виджет
}
}

```

3.3.2 Отрисовка временной шкалы

За отрисовку временной шкалы отвечает метод `drawTimeBar`, в котором рисуются такты и их номера. Внутри него также отрисовываются короткие линии для обозначения четвертей, что делает временную шкалу более информативной. Код данного метода представлен ниже.

```

void AudioTrackFrame::drawTimeBar(QPainter &painter, int width) {
    painter.setBrush(ProjectConfiguration::clTimeBar); // Устанавливаем цвет фона
временной шкалы

    int scrollOffset = parentScrollArea->verticalScrollBar()->value(); // Получаем
смещение прокрутки
    painter.drawRect(0, scrollOffset, width, TIME_BAR_HEIGHT); // Рисуем фон
временной шкалы

    painter.setPen(ProjectConfiguration::clTimeBarMark); // Устанавливаем цвет для
рисования меток времени

    // Рисуем такты и их номера
    for (int i = 0; i < trackTactCount; i++) {
        int x = i * tactDuration * scaleFactor; // Масштабируем позицию такта

        // Длинная черта для такта
        painter.drawLine(x, scrollOffset, x, TIME_BAR_HEIGHT + scrollOffset); //
Рисуем линию такта

        // Номер такта
        painter.drawText(x + 5, TIME_BAR_HEIGHT * 7 / 8 + scrollOffset,
QString::number(i)); // Рисуем номер такта

        // Короткие черточки для четвертей
        for (int j = 1; j < 4; j++) {
            int quarterX = x + j * (tactDuration / 4) * scaleFactor; // Масштабируем
позицию четверти

```

```

        painter.drawLine(quarterX, scrollOffset, quarterX, GRID_LINE_HEIGHT +
scrollOffset); // Рисуем линии для четвертей
    }
}
}

```

3.3.3 Отрисовка сетки на аудиодорожках

За рисование сетки отвечает метод `drawTrackGrid` на дорожках, который отображает линии для тактов и четвертей. Этот метод помогает организовать визуальное пространство, делая его более структурированным и удобным для восприятия. Код данного метода представлен ниже.

```

void AudioTrackFrame::drawTrackGrid(QPainter &painter, int trackCount) {
    for (int i = 0; i < trackCount; i++) {
        int y = TIME_BAR_HEIGHT + i * trackHeight; // Вычисляем вертикальную позицию
дорожки

        for (int j = 0; j < trackTactCount; j++) {
            int x = j * tactDuration * scaleFactor; // Масштабируем позицию такта

            // Линии такта
            painter.setPen(ProjectConfiguration::clAudioTrackBoldMark); //
Устанавливаем цвет для линий такта
            painter.drawLine(x, y, x, y + trackHeight); // Рисуем линию такта

            // Линии четвертей такта
            for (int k = 1; k < 4; k++) {
                int quarterX = x + k * (tactDuration / 4) * scaleFactor; //
Масштабируем позицию четверти
                painter.setPen(ProjectConfiguration::clAudioTrackMark); //
Устанавливаем цвет для линий четверти
                painter.drawLine(quarterX, y, quarterX, y + trackHeight); // Рисуем
линию для четверти
            }
        }
    }
}

```

Таким образом, метод `paintEvent` и его вспомогательные функции обеспечивают комплексную отрисовку интерфейса для работы с аудиотреками, учитывая масштабирование и текущие состояния интерфейса, что делает его удобным для пользователя.

4 ТЕСТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

В процессе тестирования было обнаружено, что во время открытия проекта логические аудиофайлы в программе содержали некорректные данные, что приводило к вылетам программы.

Проблема представлена на рисунке 4.1.

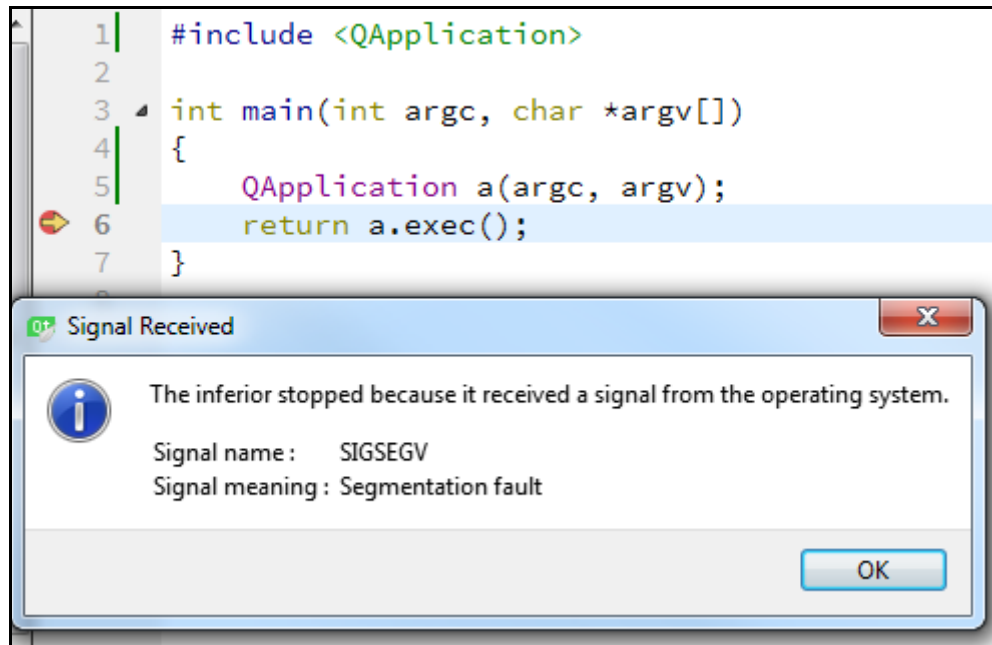


Рисунок 4.1 – Вылет программы из-за неверного обращения к памяти

Для решения этой проблемы были внесены правки в фрагмент кода, который должен был открывать проект. Внутри этого фрагмента необходимо было изменить порядок загрузки данных проекта. Фрагмент исправленного кода представлен ниже.

```
// Подключаем действие открытия проекта к обработчику события
connect(openProjectAction, &QAction::triggered, this, [&]() {

    // Открываем диалоговое окно для выбора файла проекта
    QString filePath = QFileDialog::getOpenFileName(this, "Open Project File",
QDir::homePath(), "Project Files (*.sfp)");

    // Проверяем, что файл выбран
    if (!filePath.isEmpty()) {
        QFile file(filePath); // Создаем объект QFile для работы с файлом

        // Пытаемся открыть файл только для чтения
        if (!file.open(QIODevice::ReadOnly)) {
            // Если не удалось открыть файл, создаем и отображаем сообщение об
ошибке

            QMessageBox msgBox(this);
            msgBox.setWindowTitle("Error"); // Устанавливаем заголовок окна
            msgBox.setIcon(QMessageBox::Warning); // Установка иконки
предупреждения
```

```

        msgBox.setText("Could not open file for reading."); // Текст
сообщения
        msgBox.setStandardButtons(QMessageBox::Yes | QMessageBox::No); //
Кнопки для выбора

        // Применение стиля к QMessageBox
        msgBox.setStyleSheet(
            "QPushButton { background-color: " +
ProjectConfiguration::clDarkButton.name() + "; color: " +
ProjectConfiguration::clSidePanelText.name() + "; }"
            + "QPushButton:hover { background-color: " +
ProjectConfiguration::clDarkButtonSelected.name() + "; }"
        );
        msgBox.exec(); // Отображаем сообщение
        return; // Завершаем выполнение, если файл не открылся
    }

    defaultModel->clear(); // Очищаем текущую модель проекта

    projectFullPath = filePath; // Сохраняем полный путь к проекту
    projectPath = QFileInfo(filePath).absolutePath(); // Получаем путь к
директории проекта

    QDir currDir(projectPath); // Создаем объект QDir для работы с
директорией проекта
    projectDir = currDir; // Сохраняем текущую директорию
    audioList->loadAudioFiles(projectDir.filePath("resources")); //
Загружаем аудиофайлы из папки ресурсов

    // Чтение количества дорожек из файла
    int trackCount = 0;
    if (file.read(reinterpret_cast<char*>(&trackCount), sizeof(int)) !=
sizeof(int)) {
        // Если не удалось прочитать количество дорожек, отображаем
сообщение об ошибке
        QMessageBox msgBox(this);
        msgBox.exec(); // Отображаем сообщение
        file.close(); // Закрываем файл
        return; // Выходим, если файл пустой
    }

    // Создаем дорожки в модели на основе прочитанного количества
    for (int i = 0; i < trackCount; i++) {
        AudioTrack *track = new AudioTrack(); // Создаем новый объект
AudioTrack

        track->isTrackChanged = true; // Помечаем дорожку как измененную
        defaultModel->addTrack(track); // Добавляем дорожку в модель
    }

    // Чтение данных токенов из файла
    while (!file.atEnd()) { // Пока не достигнут конец файла
        int tokenAudiofileID; // Идентификатор аудиофайла токена
        double tokenStartPositionView; // Начальная позиция токена
        double tokenDurationView; // Длительность токена
        double tokenRelativeStartTimeView; // Относительное время начала
токена

        double tokenRelativeDurationView; // Относительная длительность
токена

        int tokenAudioTrack; // Индекс дорожки токена

        // Чтение данных токена из файла

```

```

        file.read((char*)&tokenAudiofileID, sizeof(tokenAudiofileID));
        file.read((char*)&tokenStartPositionView,
sizeof(tokenStartPositionView));
        file.read((char*)&tokenDurationView, sizeof(tokenDurationView));
        file.read((char*)&tokenRelativeStartTimeView,
sizeof(tokenRelativeStartTimeView));
        file.read((char*)&tokenRelativeDurationView,
sizeof(tokenRelativeDurationView));
        file.read((char*)&tokenAudioTrack, sizeof(tokenAudioTrack));

        // Проверка, что индекс дорожки токена валиден
        if (tokenAudioTrack < 0 || tokenAudioTrack >= defaultModel-
>tracks.size()) {
            QMessageBox::warning(this, "Error", "Invalid token data (audio
track out of range)."); // Сообщение об ошибке
            break; // Прерываем цикл
        }

        // Создаем новый токен с прочитанными данными
        AudioToken* token = new AudioToken(tokenAudiofileID,
tokenStartPositionView,
tokenDurationView,
tokenAudioTrack, linker,
tokenRelativeStartTimeView, tokenRelativeDurationView);

        // Добавляем токен в соответствующую дорожку
        defaultModel->tracks[token->audioTrack]->tokens.append(*token);
    }

    file.close(); // Явно закрываем файл
}
emit projectCreated(); // Генерируем сигнал о создании проекта
});

```

Этот код обрабатывает событие открытия проекта, позволяя пользователю выбрать файл с расширением .sfp. После выбора файла он пытается открыть его для чтения. Если открытие успешно, очищается текущая модель проекта и загружаются аудиофайлы из директории проекта. Затем считывается количество дорожек, создаются новые дорожки в модели и читаются данные токенов, которые проверяются на корректность и добавляются в соответствующие дорожки. В конце файл закрывается, и генерируется сигнал о создании проекта.

5 РУКОВОДСТВО ПО УСТАНОВКЕ И ИСПОЛЬЗОВАНИЮ ПРОГРАММНОГО СРЕДСТВА

5.1 Взаимодействие с программным средством

5.1.1 Создание проекта

После запуска программы открывается главное окно. В таком состоянии все кнопки находятся в заблокированном состоянии до тех пор, пока проект не будет создан. Для того чтобы создать проект необходимо выбрать пункт главного меню File => Create project. Затем в открывшемся диалоговом окне необходимо выбрать корневую папку проекта. После этого внутри нее программа создаст файл проекта *.sfp (sound forge project) и пустую папку resources для хранения ресурсов проекта.

В созданном проекте кнопки становятся доступными. В проекте можно добавить пустые аудиодорожки и аудиотокены на них. Внешний вид главного окна приведен на рисунке 5.1.

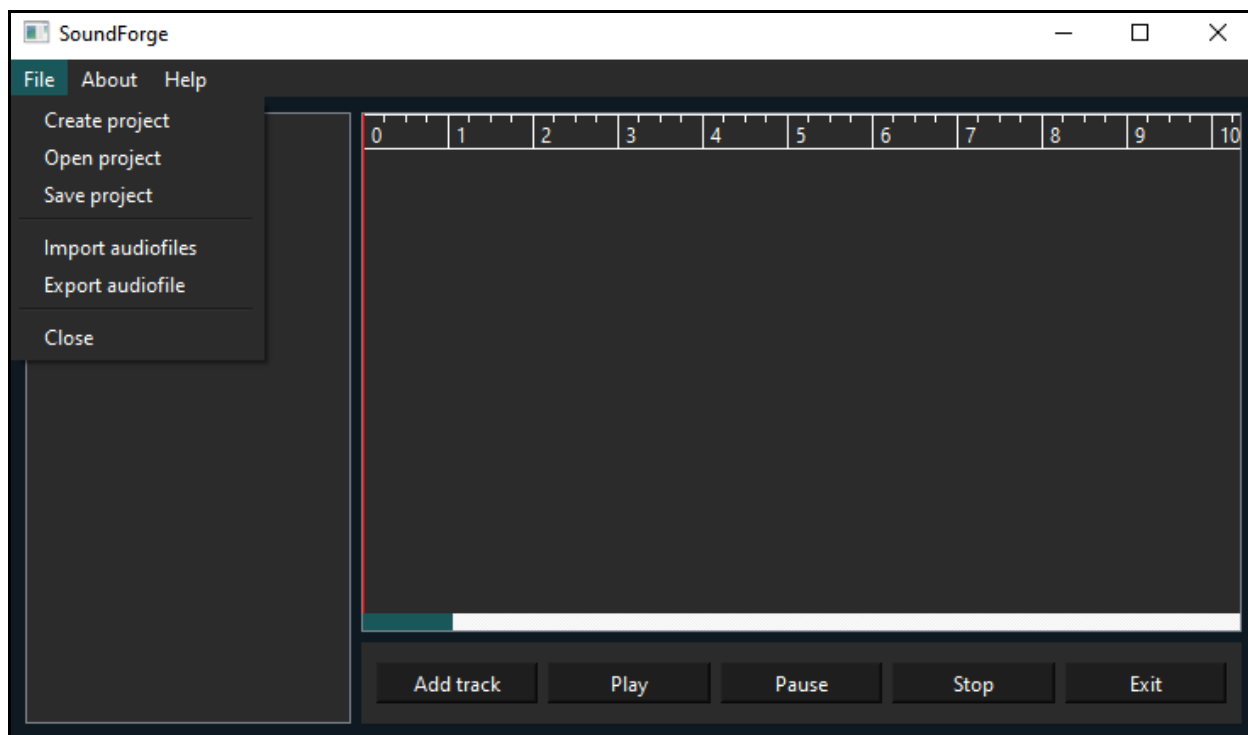


Рисунок 5.1 – главное окно нового проект

5.1.2 Импорт аудиофайлов

После создания проекта необходимо импортировать аудиофайлы для их обработки. Для этого необходимо выбрать пункт меню File => Import audiofiles, а затем выбрать импортируемые файлы для загрузки.

После загрузки файлов можно приступить к обработке. Внешний вид главного окна после импорта приведен на рисунке 5.2.

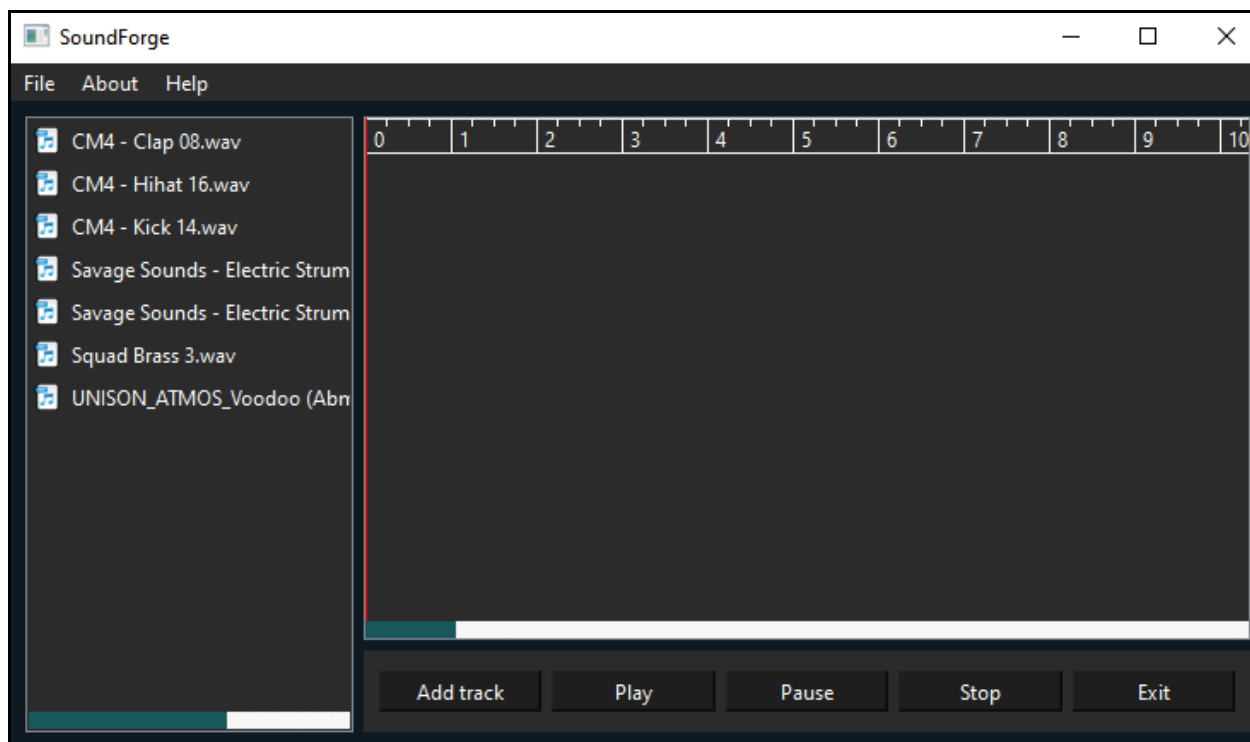


Рисунок 5.2 – главное окно после импорта аудиофайлов

5.1.3 Создание аудиодорожек и размещение сэмплов на них

После загрузки аудиофайлов необходимо добавить в рабочую область одну или несколько аудиодорожек. Затем из списка загруженных аудиофайлов можно перетащить нужный файл на аудиодорожки, захватив файл левой клавишей мыши.

Размещенные токены также можно передвигать по аудиодорожкам. Токены перемещаются по сетке, а с зажатой клавишей Shift их можно двигать без привязки к сетке.

Если токен не понадобился – его можно удалить, нажав правую клавишу мыши и выбрав пункт меню Delete token.

Внешний вид главного окна после размещения токенов приведен на рисунке 5.3.

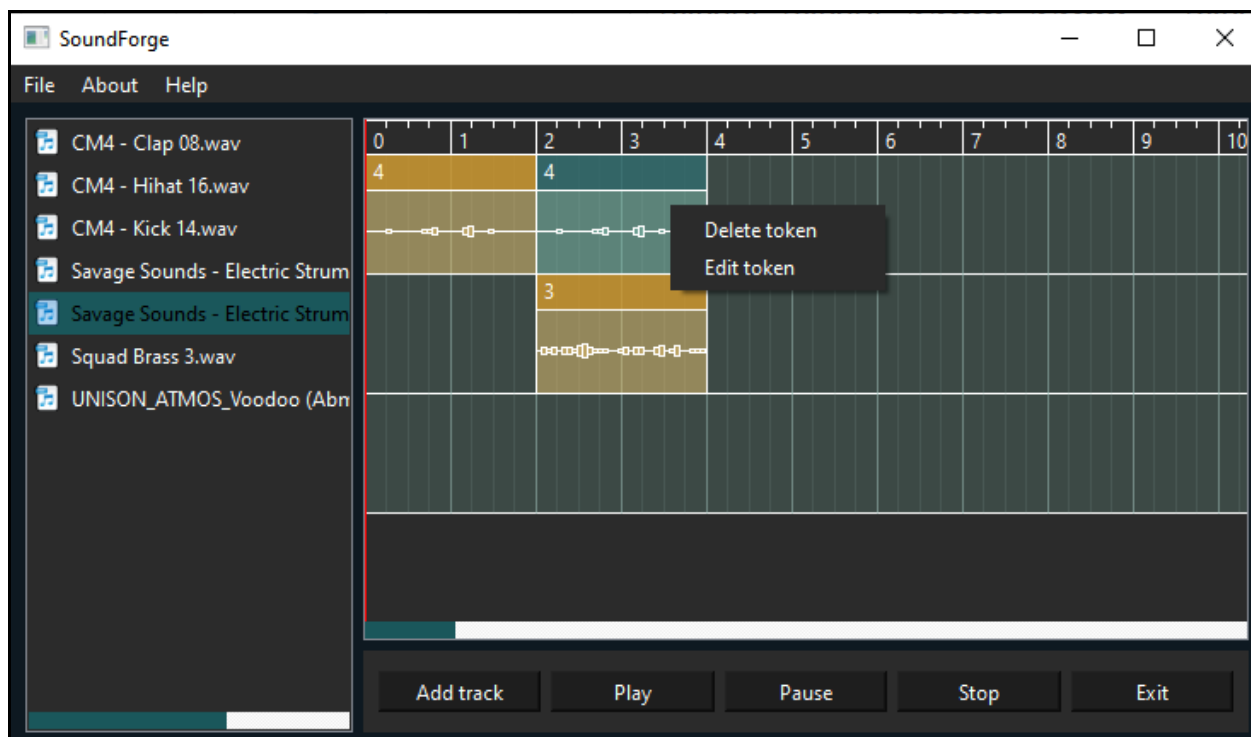


Рисунок 5.3 – главное окно после размещения токенов

5.1.4 Редактирование отдельного токена

Для более гибкого управления аудиотокенами пользователь может открыть окно редактирования конкретного сэмпла нажав правую клавишу мыши и выбрав пункт меню Edit token.

Внутри открывшегося окна с помощью инструмента slice пользователь может разделить выбранный токен на несколько частей, удалить ненужные части и применить изменения, нажав кнопку Confirm или отклонить изменения, нажав Cancel.

Внешний вид окна редактирования аудиотокена приведен на рисунке 5.4.

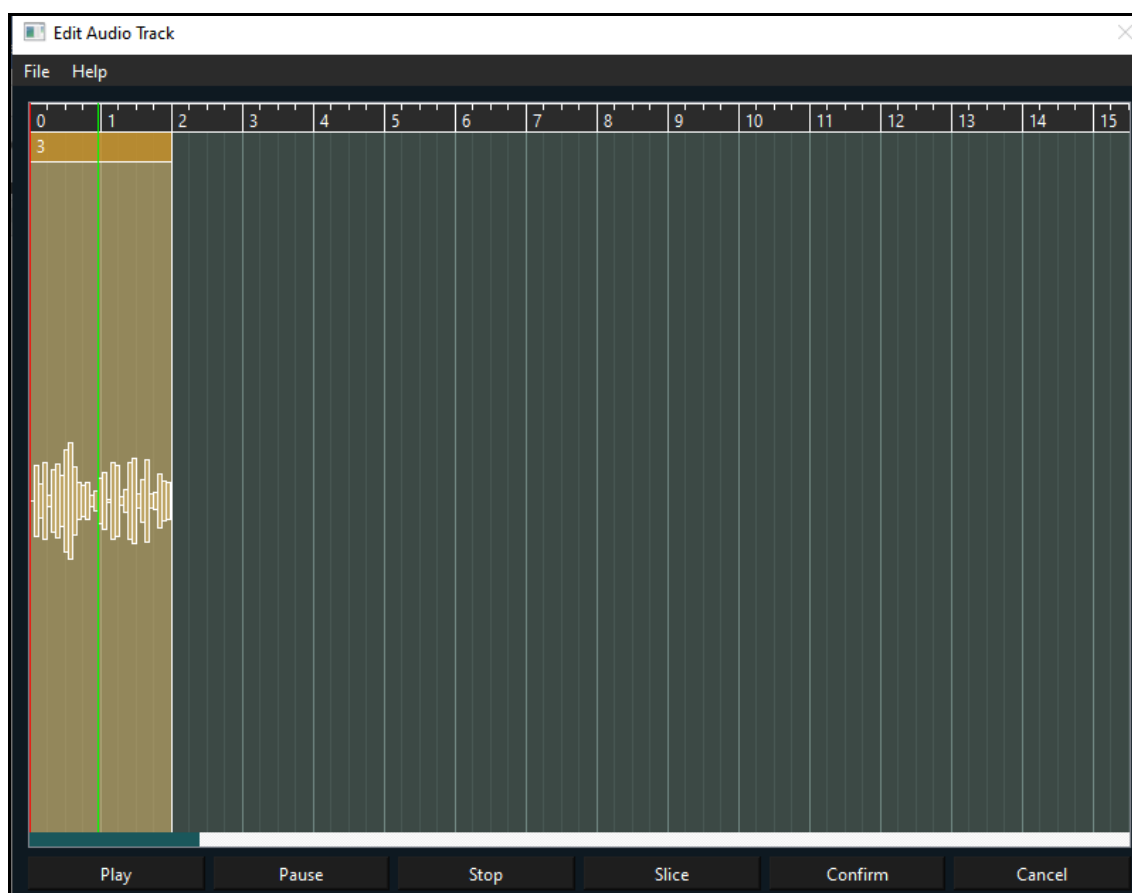


Рисунок 5.4 – окно редактирования аудиотокена

Внутри данного окна добавлены кнопки Play, Pause и Stop для управления проигрыванием отредактированного токена.

ЗАКЛЮЧЕНИЕ

В рамках курсового проекта было разработано программное средство— аудиоредактор SoundForge. Приложение обладает интуитивно-понятным графическим интерфейсом, который упрощает взаимодействие пользователей с функционалом программы. Особое внимание уделялось тщательному тестированию, в результате которого были выявлены и исправлены все ошибки и недочеты, что обеспечило высокую производительность и надежность работы приложения.

Аудиоредакторы играют ключевую роль в современном музыкальном производстве и обработке звука. Они предоставляют пользователям широкий спектр инструментов для редактирования, микширования и создания аудиотреков, что позволяет реализовывать творческие идеи и достигать профессиональных результатов. Современные аудиоредакторы также интегрируют функции для работы с MIDI, эффектами и виртуальными инструментами, что значительно расширяет их возможности.

В будущем планируется дальнейшее развитие SoundForge. Среди предполагаемых улучшений — добавление инструментов для копирования токенов, что упростит работу с аудиотреками, а также оптимизация работы с памятью для повышения общей эффективности приложения. Так же в приложении планируется создать основу для подключения различных плагинов. Эти улучшения сделают SoundForge более универсальным и полезным для пользователей, расширяя его функциональные возможности и улучшая опыт работы с аудиоданными.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

- [1] Орлов, С. А. Технологии разработки программного обеспечения: учеб. Пособие. – СПб, 2003. – 321 с.
- [2] Майкл Д. Изучаем C++ через программирование игр. – СПб.: Питер. 2015. – 450с.
- [3] Лафоре Р. Объектно-ориентированное программирование в C++. 4-е полное изд. – СПб.: Питер, 2018. – 928 с.
- [4] Мартин Р. Чистый код: создание, анализ и рефакторинг. Библиотека программиста. - СПб.: Питер. 2018. – 464 с.
- [5] Уилсон, С. Принципы проектирования и разработки программного обеспечения, учебн. курс. – СПб, 2003. – 570 с.
- [6] <https://doc.qt.io/> – Документация к Qt.
- [7] Петцольд 4. Programming Windows®, 1998. - СПб.: БХВ-Петербург, 2007. – 1448 с.
- [8] Пол А. Объектно-ориентированное программирование на C++. – Москва, 2017. – 476с.

ПРИЛОЖЕНИЕ А

Исходный код программы

```
#ifndef PROJECTCONFIGURATION_H
#define PROJECTCONFIGURATION_H

#include <QColor>
#include <QString>

//MainWindow
const int MIN_MAIN_WINDOW_WIDTH = 600;
const int MIN_MAIN_WINDOW_HEIGHT = 400;

//AudioFileObject
const double MS_TO_PX = 75;

//AudioList
const float DEFAULT_AUDIO_LIST_RELATIVE_WIDTH = 0.3;
const int DEFAULT_LIST_ITEM_HEIGHT = 25;
const int DEFAULT_SCROLLBAR_SIZE = 10;
const QString MP3_ICON_PATH = "F:/Projects/SoundForge/icons/mp3-icon.ico";
const QString AUDIO_RESOURCES_PATH = "F:/Projects/SoundForge/AudioFiles";
const QString DEFAULT_PROJECT_NAME = "soundforge.sfp";

//AudioTrackFrame
const int TRACK_HEIGHT = 70;
const int DELTA = 2;
const int DEFAULT_TACT_DURATION = 50;
const int DEFAULT_TACT_COUNT = 100;

const int TIME_BAR_HEIGHT = 20;
const int GRID_LINE_HEIGHT = 4; // Высота короткой черты для четвертей такта

const double MIN_SCALE = 0.4;
const double MAX_SCALE = 10.0;

//AudioTrack
const int SAMPLE_RATE = 44100; // Частота дискретизации (Гц)
const int CHANNELS = 2; // Количество каналов (стерео)
const int BITS_PER_SAMPLE = 16; // Битность (16 бит)
const int BLOCK_ALIGN = CHANNELS * (BITS_PER_SAMPLE / 8); // Размер блока данных
const double MS_TO_BYTES = SAMPLE_RATE * BLOCK_ALIGN / 1000.0; // Перевод миллисекунд
// в байты
const int WAV_HEADER_SIZE = 44;

//AudioToken
const double TOKEN_HEADER_RELATIVE_HEIGHT = 0.30;

//ControlPanel
const float DEFAULT_CONTROL_PANEL_RELATIVE_HEIGHT = 0.1;
const int ADD_TRACK_BUTTON_WIDTH = 100;

//Dialog
const int DIALOG_TRACK_HEIGHT = 500;

//Colors
const QColor DARK_COLOR = QColor::fromRgb(0x1e, 0x1e, 0x1e);
const QColor DARK_GRAY_COLOR = QColor::fromRgb(0x2b, 0x2b, 0x2b);
const QColor WHITE_COLOR = QColor::fromRgb(0xff, 0xff, 0xff);
const QColor BLACK_COLOR = QColor::fromRgb(0x0e, 0x19, 0x21);
const QColor BLUE_COLOR = QColor::fromRgb(0x1a, 0x57, 0x5b);
const QColor SCROLLBAR_BACKGROUND_COLOR = QColor::fromRgb(0xf0, 0xf0, 0xf0);
const QColor LIGHT_GRAY_COLOR = QColor::fromRgb(0x3c, 0x49, 0x45);
const QColor LIGHT_BLUE_COLOR = QColor::fromRgb(0xac, 0xcf, 0xc7, 30);
const QColor LIGHT_BLUE_BOLD_COLOR = QColor::fromRgb(0xac, 0xcf, 0xc7, 120);

const QColor DARK_DRAGGED_TOKEN_COLOR = QColor::fromRgb(0xfd, 0x4c, 0x3c, 120);
```

```

const QColor LIGHT_DRAGGED_TOKEN_COLOR = QColor::fromRgb(0xf3, 0x7e, 0x63, 70);

const QColor DARK_ORANGE_TOKEN_COLOR = QColor::fromRgb(0xd8, 0x9b, 0x2b, 200);
const QColor LIGHT_ORANGE_TOKEN_COLOR = QColor::fromRgb(0xe8, 0xc2, 0x6f, 130);

const QColor DARK_TEAL_TOKEN_COLOR = QColor::fromRgb(0x30, 0x6c, 0x6f, 200);
const QColor LIGHT_TEAL_TOKEN_COLOR = QColor::fromRgb(0x7b, 0xbb, 0xab, 130);

const QColor SELECTED_TRACK_COLOR = QColor::fromRgb(0x7b, 0xbb, 0xab, 130);

class ProjectConfiguration
{
public:
    ProjectConfiguration();

    static QColor clAudioTrackFrame;
    static QColor clSidePanel;
    static QColor clSidePanelText;
    static QColor clControlPanel;
    static QColor clSidePanelItemSelected;
    static QColor clBackground;
    static QColor clDarkButton;
    static QColor clDarkButtonSelected;
    static QColor clMenuBarItemSelected;
    static QColor clScrollbarBackground;
    static QColor clScrollbar;
    static QColor clTimeBar;
    static QColor clTimeBarMark;
    static QColor clAudioTrack;
    static QColor clSelectedAudioTrack;
    static QColor clAudioTrackMark;
    static QColor clAudioTrackBoldMark;
    static QColor clDefaultTokenHeader;
    static QColor clDefaultTokenMainContent;
    static QColor clSelectedAudioTokenHeader;
    static QColor clSelectedAudioTokenMainContent;
    static QColor clTokenText;
};

#endif // PROJECTCONFIGURATION_H

#ifndef AUDIOTRACKFRAME_H
#define AUDIOTRACKFRAME_H

#include <QFrame>
#include "workspace-implementation/audiotoken.h"
#include "workspace-implementation/audiotrack.h"
#include "workspace-implementation/workspacemodel.h"
#include <QScrollArea>
#include "audiofilelinker.h"

class AudioTrackFrame : public QFrame {
    Q_OBJECT

public:
    AudioToken *draggedToken;
    bool isTokenDragging;
    bool isCurrTimeChanging;
    double draggedTokenStartX; //такие значения храним при масштабе 1.0
    double draggedTokenStartY;
    int draggedTokenIndex;
    double draggedTokenDeltaX; //расстояние от startPosition до точки захвата
    double draggedTokenDeltaY; //расстояние от startPosition до точки захвата
    WorkspaceModel *model;
    int trackTactCount;
    double tactDuration;
    int trackHeight;
    bool isSlicing;
    int slicingX;

```

```

    AudioTrack *totalTrack;
    explicit AudioTrackFrame(int trackHeight = TRACK_HEIGHT, QWidget *parent =
nullptr, QScrollArea *scrollArea = nullptr, AudioFileLinker *linker = nullptr);
    void setModel(WorkspaceModel *model);
    WorkspaceModel* getModel();
    void mousePressEvent(QMouseEvent *event) override;
    void mouseMoveEvent(QMouseEvent *event) override;
    void mouseReleaseEvent(QMouseEvent *event) override;
    void paintEvent(QPaintEvent *event) override;
    void resizeToFitContent();
    void wheelEvent(QWheelEvent *event) override;
    void showTrackContextMenu(const QPoint &pos, int trackIndex);
    void showTokenContextMenu(const QPoint &pos, int trackIndex, int tokenIndex);
    void deleteTrack(int trackIndex);
    void deleteToken(int trackIndex, int tokenIndex);
    void openEditTokenWindow(int trackIndex, int tokenIndex);
    void dragEnterEvent(QDragEnterEvent *event);
    void dragMoveEvent(QDragMoveEvent *event);
    void dropEvent(QDropEvent *event);
    bool hasSomeChanges();
    void playAudioTokens(); // Метод для запуска воспроизведения
    QByteArray mixWavFiles(QByteArray *wav1, QByteArray *wav2);

protected:
    void drawTimeBar(QPainter &painter, int width);
    void drawTrackGrid(QPainter &painter, int trackCount);

public slots:
    void onTrackAdded(); // Slot to handle track addition
    void onPlayClicked();
    void onPauseClicked();
    void onStopClicked();
    void onExitClicked();
    void onVerticalScrollBarChanged();
    void onCurrTimeChanged();
    void onTrackChanged(int index);
    void onAudioFileFinished();

signals:
    void currTimeChanged();
    void timeBarClicked();
    void trackChanged(int index);

private:
    QScrollArea* parentScrollArea;
    AudioFileLinker *fileLinker;
    double scaleFactor; // Масштабирование
    QTimer *timer; // таймер для отслеживания времени
    int startTime; // время начала воспроизведения в миллисекундах
    qint64 currTime;
    int currViewTime;
    int startViewTime;

    void updateCurrTime(); // метод для обновления currTime
};

#endif // AUDIOTRACKFRAME_H

#ifndef AUDIOTOKEN_H
#define AUDIOTOKEN_H

#include <QPainter>
#include "../projectconfiguration.h"
#include "../audiofilelinker.h"
class AudioToken
{
public:

```

```

        AudioToken(int audiofileID, double startPosition, double duration, int audioTrack,
                    AudioFileLinker *linker, double relStartPosition, double relDuration,
                    QColor headerColor = ProjectConfiguration::clDefaultTokenHeader,
                    QColor mainContentColor =
ProjectConfiguration::clDefaultTokenMainContent);

        AudioToken(int audiofileID, double startPosition, double duration, int audioTrack,
                    AudioFileLinker *linker,
                    QColor headerColor = ProjectConfiguration::clDefaultTokenHeader,
                    QColor mainContentColor =
ProjectConfiguration::clDefaultTokenMainContent);

        int tokenID;
        int audiofileID;           //ID аудиофайла
        double startPositionView;   //начало фрагмента на аудиодорожке в px
        double durationView;        //длительность фрагмента в px
        double relativeStartTimeView; //начало активной части фрагмента относительно всего
фрагмента в px
        double relativeDurationView; //длительность активной части фрагмента в px
        int audioTrack;             //номер аудиодорожки
        bool isSelected;
        static int idCounter;
        AudioFileLinker *linker;

        void drawToken(QPainter *painter, double scaleFactor, const int trackHeight)
const;
        void updateTokenRelativeStartPosition(int newStartTime);
        void updateTokenRelativeDuration(double newDuration);
        void resetSelectedFlag();
        void setSelectedFlag();
private:
        QColor headerColor;
        QColor mainContentColor;

};

#endif // AUDIOTOKEN_H

#ifndef AUDIOTRACK_H
#define AUDIOTRACK_H

#include <QList>
#include "audiotoken.h"
#include <QByteArray>
#include <QMediaPlayer>
#include <QBuffer>
#include <QAudioOutput>

class AudioTrack : public QObject
{
    Q_OBJECT

public:
    explicit AudioTrack(QObject *parent = nullptr);

    // Удаляем конструктор копирования и оператор присваивания
    AudioTrack(const AudioTrack&) = delete;
    AudioTrack& operator=(const AudioTrack&) = delete;

    void addToken(AudioToken &token);
    QList<AudioToken>& getTokens();
    QList<AudioToken> tokens;
    QByteArray *trackData;
    QBuffer *buffer;
    QMediaPlayer *player;
    QAudioOutput *audioOutput;
    bool isTrackChanged;
    bool isSelected;
    void processAudioTrack();

```

```

        void playTrack(qint64 startTime);
        void pauseTrack();
        void setTrackChangeFlag();
        void clearData();

private slots:
    void onMediaStatusChanged(QMediaPlayer::MediaStatus status);

signals:
    void audioFileFinished();

};

#endif // AUDIOTRACK_H

#ifndef WORKSPACEMODEL_H
#define WORKSPACEMODEL_H

#include <QAbstractListModel>
#include <QList>
#include <QVariant>
#include "audiotrack.h"

class WorkspaceModel : public QAbstractListModel
{
    Q_OBJECT

public:
    explicit WorkspaceModel(QObject *parent = nullptr);

    int rowCount(const QModelIndex &parent = QModelIndex()) const override;
    QVariant data(const QModelIndex &index, int role = Qt::DisplayRole) const
    override;

    void addTrack(AudioTrack *track);
    void moveToken(int oldTrackIndex, int newTrackIndex, int tokenIndex, double
    newStartPosition);
    void clear();

    QList<AudioTrack*> &getTracks();
    QList<AudioTrack*> tracks;

};

#endif // WORKSPACEMODEL_H

#ifndef AUDIOFILELINKER_H
#define AUDIOFILELINKER_H

#include <QList>
#include <QString>
#include "audiofileobject.h"

class AudioFileLinker
{
public:
    static QList<AudioFileObject*> audioFiles; // Используем указатели
    AudioFileLinker();

    int appendAudioFile(const QString &filePath);
    int getLastID() const;
    QString getFilePathByID(int ID) const;
    static double calculateDuration(int ID);
};

#endif // AUDIOFILELINKER_H

```

```

#ifndef AUDIOFILELIST_H
#define AUDIOFILELIST_H

#include <QListView>
#include <QStandardItemModel>
#include <QDir>
#include <QDragEnterEvent>
#include <QDropEvent>
#include <QMimeType>
#include "audiofilelinker.h"

class AudioFileList : public QListView {
    Q_OBJECT

public:
    AudioFileList(QString path = "", QWidget *parent = nullptr, AudioFileLinker
*linker = nullptr);
    AudioFileLinker *audioFileLinker;
    void loadAudioFiles(const QString &directoryPath);
    QString dirPath;
    QStandardItemModel *model;

protected:
    void startDrag(Qt::DropActions supportedActions) override;
    void dragEnterEvent(QDragEnterEvent *event) override;
    void dragMoveEvent(QDragMoveEvent *event) override;
    void dropEvent(QDropEvent *event) override;

private:
    void removeEmptyRows();
};

#endif // AUDIOFILELIST_H

#ifndef AUDIOFILEOBJECT_H
#define AUDIOFILEOBJECT_H

#include <QString>
#include <QByteArray>
#include <QAudioFormat>
#include <QMediaPlayer>
#include <QMediaMetaData>
#include <QDebug>
#include <QFile>
#include <QUrl>
#include <QObject>

class AudioFileObject : public QObject {
    Q_OBJECT

public:
    // Конструкторы
    explicit AudioFileObject(QObject *parent = nullptr);
    AudioFileObject(const QString &filePath, QObject *parent = nullptr);

    // Методы
    bool loadAudioFile(const QString &filePath); // Загрузка аудио
    void analyzeAudio(); // Анализ аудиофайла
    void printAudioInfo() const; // Вывод информации об аудио

    // Поля
    bool canAccess; // Флаг, доступен ли файл
    bool isEmpty; // Флаг, пустой ли объект
    QString audioFilePath; // Путь к аудиофайлу
    QByteArray audioData; // Указатель на данные аудиофайла
    qint64 durationMs; // Длительность аудио в миллисекундах
    int sampleRate; // Частота дискретизации (Гц)

```

```

        int bitRate;                                // Битрейт (бит/с)
        int channels;                                // Количество каналов (моно/стерео)

public slots:
    void onMediaStatusChanged(QMediaPlayer::MediaStatus status);

private:
    QMediaPlayer mediaPlayer;

};

#endif // AUDIOFILEOBJECT_H

#ifndef AUDIOTRACKDIALOG_H
#define AUDIOTRACKDIALOG_H

#include <QDialog>
#include <QPainter>
#include <QMouseEvent>
#include <QVBoxLayout>
#include <QScrollArea>
#include <QPushButton>
#include <QMenuBar>
#include <QMenu>
#include <QScrollBar>
#include <QVector>
#include "projectconfiguration.h"
#include "workspace-implementation/audiotoken.h"
#include "workspace-implementation/audiotrack.h"
#include "workspace-implementation/workspacemodel.h"
#include "audiotrackframe.h"
#include "QWidget"
#include <QFileDialog>
#include <QFile>
#include <QMessageBox>
#include <QLabel>

class AudioTrackDialog : public QDialog {
    Q_OBJECT

public:
    explicit AudioTrackDialog(WorkspaceModel *baseModel, int trackIndex, int
tokenIndex, AudioFileLinker *linker, QWidget *parent = nullptr);
    QList<AudioToken> dialogTokens;

private:
    void confirmChanges();
    void cancelChanges();

private:
    AudioTrackFrame* dialogTrackFrame;
    WorkspaceModel* baseModel;
    AudioToken *editingToken;
    AudioTrack *dialogTrack;

public slots:
    void onSliceButtonClicked();
};

#endif // AUDIOTRACKDIALOG_H

#ifndef CONTROLPANEL_H
#define CONTROLPANEL_H

#include <QFrame>
#include <QPushButton>
#include <QHBoxLayout>
#include "audiotrackframe.h"

```



```

class ControlPanel : public QFrame {
    Q_OBJECT

public:
    ControlPanel(AudioTrackFrame *trackframe, QWidget *parent = nullptr);

private:
    QPushButton *playButton;
    QPushButton *pauseButton;
    QPushButton *stopButton;
    QPushButton *exitButton;
    QPushButton *addTrackButton;
    AudioTrackFrame* trackFrame;

public slots:
    void onAddTrackButtonClicked(); // Slot for button click
    void onPlayButtonClicked();
    void onPauseButtonClicked();
    void onStopButtonClicked();
    void onExitButtonClicked();
    void onProjectCreated();
signals:
    void trackAdded();
    void playClicked();
    void pauseClicked();
    void stopClicked();
    void exitClicked();
};

#endif // CONTROL_PANEL_H

#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QHBoxLayout>
#include <QMenuBar>
#include <QMenu>
#include <QLabel>
#include <QDir>
#include "audiofilelinker.h"
#include "audiotrackframe.h"
#include "audiofilelist.h"
#include "controlpanel.h"

class MainWindow : public QMainWindow {
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();
    QDir projectDir;
    QString projectPath;
    QString projectFullPath;
    AudioFileLinker *linker;
    AudioTrackFrame *trackFrame;
    WorkspaceModel *defaultModel;
    QScrollArea *scrollArea;
    AudioFileList *audioList;
    ControlPanel *controlPanel;

    bool isProjectCreated;
public slots:
    void onProjectCreated();

private:
    void setupUI();

```

```

signals:
    void projectCreated();
};

#endif // MAINWINDOW_H

#include "audiotrackframe.h"
#include "projectconfiguration.h"
#include "workspace-implementation/workspacemodel.h"
#include "audiofilelinker.h"
#include "controlpanel.h"
#include "audiotrackdialog.h"
#include <QPainter>
#include <QMouseEvent>
#include <cmath>
#include <QScrollBar>
#include <QMimeType>
#include <QDrag>
#include <QTimer>
#include <QDateTime>
#include <QAudioOutput>
#include <QBuffer>
#include <QMenu>
#include <QMenuBar>
#include <QDialog>
#include <QMessageBox>

AudioTrackFrame::AudioTrackFrame(int trackHeight, QWidget *parent, QScrollArea*
scrollArea, AudioFileLinker *linker)
    : QFrame(parent),
    draggedToken(nullptr),
    isTokenDragging(false),
    isCurrTimeChanging(false),
    draggedTokenStartX(0),
    draggedTokenStartY(0),
    draggedTokenIndex(-1),
    currTime(0),
    currViewTime(0),
    startViewTime(0),
    trackTactCount(DEFAULT_TACT_COUNT),
    tactDuration(DEFAULT_TACT_DURATION),
    parentScrollArea(scrollArea),
    fileLinker(linker),
    scaleFactor(1.0),
    model(nullptr),
    trackHeight(trackHeight),
    isSlicing(false),
    slicingX(0)
{
    setAcceptDrops(true);
    setMouseTracking(true);

    timer = new QTimer(this);
    totalTrack = new AudioTrack();

    connect(scrollArea->verticalScrollBar(), &QScrollBar::valueChanged, this,
    &AudioTrackFrame::onVerticalScrollBarChanged);
    connect(this, &AudioTrackFrame::currTimeChanged, this,
    &AudioTrackFrame::onCurrTimeChanged);
    connect(this, &AudioTrackFrame::timeBarClicked, this,
    &AudioTrackFrame::onPauseClicked);
    connect(this, &AudioTrackFrame::trackChanged, this,
    &AudioTrackFrame::onTrackChanged);
    connect(timer, &QTimer::timeout, this, &AudioTrackFrame::updateCurrTime);
}

void AudioTrackFrame::setModel(WorkspaceModel *model) {
    this->model = model;
    resizeToFitContent();
    update();
}

```

```

}

WorkspaceModel* AudioTrackFrame::getModel() {
    return model;
}

void AudioTrackFrame::paintEvent(QPaintEvent *event) {
    QFrame::paintEvent(event);
    if (!model) return;

    QPainter painter(this);
    int trackWidth = trackTactCount * tactDuration * scaleFactor; // Масштабируем
    ширину трека

    // Рисуем дорожки
    for (int i = 0; i < model->rowCount(); i++) {
        AudioTrack *track = model->tracks[i];

        // Фон дорожки
        painter.setBrush((track->isSelected) ?
ProjectConfiguration::clSelectedAudioTrack : ProjectConfiguration::clAudioTrack);
        painter.drawRect(0, TIME_BAR_HEIGHT + i * trackHeight, trackWidth,
trackHeight);
    }

    // Рисуем сетку для дорожек (каждые 1/4 такта)
    drawTrackGrid(painter, model->rowCount());

    // Рисуем токены на дорожке
    for (int i = 0; i < model->rowCount(); i++) {
        AudioTrack *track = model->tracks[i];

        if (isTokenDragging && draggedToken->audioTrack == i) {
            for (int j = 0; j < track->tokens.size(); j++) {
                if (draggedTokenIndex != j) {
                    AudioToken token = track->tokens.at(j);
                    token.drawToken(&painter, scaleFactor, trackHeight); // Передаем
масштаб
                }
            }
        } else {
            for (AudioToken token : track->tokens) {
                token.drawToken(&painter, scaleFactor, trackHeight); // Передаем
масштаб
            }
        }

        // Рисуем перетаскиваемый токен
        if (isTokenDragging) {
            painter.setBrush(Qt::red);
            int x = draggedTokenStartX;
            int targetTrack = (draggedTokenStartY) / trackHeight;
            if (targetTrack >= 0 && targetTrack < model->rowCount() && x >= 0) {
                int headerHeight = TRACK_HEIGHT * TOKEN_HEADER_RELATIVE_HEIGHT;
                int mainContentHeight = trackHeight - headerHeight;

                // Верхняя часть токена с текстом
                painter.setPen(ProjectConfiguration::clTokenText);
                painter.setBrush(DARK_DRAGGED_TOKEN_COLOR);
                painter.drawRect(x, TIME_BAR_HEIGHT + targetTrack * trackHeight,
draggedToken->relativeDurationView * scaleFactor, headerHeight);

                // Нижняя часть для будущего отображения аудиосигнала
                painter.setBrush(LIGHT_DRAGGED_TOKEN_COLOR);
                painter.drawRect(x, TIME_BAR_HEIGHT + targetTrack * trackHeight +
headerHeight, draggedToken->relativeDurationView * scaleFactor, mainContentHeight);
            }
        }
    }
}

```

```

        // Рисуем временную шкалу
        drawTimeBar(painter, trackWidth);

        // Рисуем текущую позицию проигрывания
        painter.setPen(Qt::red);
        painter.drawLine(currViewTime * scaleFactor, 0, currViewTime * scaleFactor,
height()); // Учет масштаба

        if (isSlicing) {
            painter.setPen(Qt::green);
            painter.drawLine(slicingX * scaleFactor, 0, slicingX * scaleFactor, height());
// Учет масштаба
            update();
        }
    }

void AudioTrackFrame::drawTimeBar(QPainter &painter, int width) {
    painter.setBrush(ProjectConfiguration::clTimeBar);

    int scrollOffset = parentScrollArea->verticalScrollBar()->value();
    painter.drawRect(0, scrollOffset, width, TIME_BAR_HEIGHT);

    painter.setPen(ProjectConfiguration::clTimeBarMark);

    // Рисуем такты и их номера
    for (int i = 0; i < trackTactCount; i++) {
        int x = i * tactDuration * scaleFactor; // Масштабируем такты

        // Длинная черта для такта
        painter.drawLine(x, scrollOffset, x, TIME_BAR_HEIGHT + scrollOffset);

        // Номер такта
        painter.drawText(x + 5, TIME_BAR_HEIGHT * 7 / 8 + scrollOffset,
QString::number(i));

        // Короткие черточки для четвертей
        for (int j = 1; j < 4; j++) {
            int quarterX = x + j * (tactDuration / 4) * scaleFactor;
            painter.drawLine(quarterX, scrollOffset, quarterX, GRID_LINE_HEIGHT +
scrollOffset);
        }
    }
}

void AudioTrackFrame::drawTrackGrid(QPainter &painter, int trackCount) {
    for (int i = 0; i < trackCount; i++) {
        int y = TIME_BAR_HEIGHT + i * trackHeight;

        for (int j = 0; j < trackTactCount; j++) {
            int x = j * tactDuration * scaleFactor; // Масштабируем такты

            // Линии такта
            painter.setPen(ProjectConfiguration::clAudioTrackBoldMark);
            painter.drawLine(x, y, x, y + trackHeight);

            // Линии четвертей такта
            for (int k = 1; k < 4; k++) {
                int quarterX = x + k * (tactDuration / 4) * scaleFactor;
                painter.setPen(ProjectConfiguration::clAudioTrackMark);
                painter.drawLine(quarterX, y, quarterX, y + trackHeight);
            }
        }
    }
}

void AudioTrackFrame::mousePressEvent(QMouseEvent *event) {
    if (event->button() == Qt::LeftButton) {
        if (!isSlicing) {
            QPoint mousePos = event->pos();

```

```

        // Клик по таймбару
        int scrollOffset = parentScrollArea->verticalScrollBar()->value();
        if (mousePos.y() > scrollOffset && mousePos.y() < scrollOffset +
TIME_BAR_HEIGHT) {
            currViewTime = mousePos.x() / scaleFactor;          // currTime при 1.0
            emit currTimeChanged();
            emit timeBarClicked();
            isCurrTimeChanging = true;
            update();
            return;
        }

        int xClick = mousePos.x();
        int yClick = mousePos.y() - TIME_BAR_HEIGHT;

        int trackIndex = yClick / trackHeight;
        if (trackIndex < 0 || trackIndex >= model->rowCount()) return;

        AudioTrack *track = model->tracks[trackIndex];
        for (int tokenIndex = 0; tokenIndex < track->tokens.size(); ++tokenIndex)
        {
            AudioToken token = track->tokens[tokenIndex];
            double x = token.startPositionView * scaleFactor;
            double w = token.relativeDurationView * scaleFactor;

            if (xClick > x && xClick < (x + w)) {
                isTokenDragging = true;
                draggedToken = new AudioToken(token);
                draggedTokenDeltaX = xClick - token.startPositionView *
scaleFactor; // startPosition при 1.0
                draggedTokenDeltaY = yClick - token.audioTrack * trackHeight;

                draggedTokenStartX = xClick - draggedTokenDeltaX; // startX и
deltaX в больших координатах
                draggedTokenStartY = yClick - draggedTokenDeltaY;
                draggedTokenIndex = tokenIndex;
                break;
            }
        }
    }
} else if (event->button() == Qt::RightButton) {
    QPoint mousePos = event->pos();
    int yClick = mousePos.y() - TIME_BAR_HEIGHT;
    int trackIndex = yClick / trackHeight;

    if (trackIndex < 0 || trackIndex >= model->rowCount()) return;

    AudioTrack *track = model->tracks[trackIndex];

    // Клик по токену
    for (int tokenIndex = 0; tokenIndex < track->tokens.size(); ++tokenIndex) {
        AudioToken token = track->tokens[tokenIndex];
        double x = token.startPositionView * scaleFactor;
        double w = token.relativeDurationView * scaleFactor;

        if (mousePos.x() > x && mousePos.x() < (x + w)) {
            showTokenContextMenu(mousePos, trackIndex, tokenIndex);
            return;
        }
    }

    // Клик по дорожке (не по токену)
    showTrackContextMenu(mousePos, trackIndex);
}

}

void AudioTrackFrame::mouseMoveEvent(QMouseEvent *event) {
    QPoint mousePos = event->pos();

    if (isCurrTimeChanging) {

```

```

        if (mousePos.x() > 0 && mousePos.x() < trackTactCount * tactDuration *
scaleFactor) {
            currViewTime = mousePos.x() / scaleFactor; // Учет масштаба
//currTime при 1.0
            emit currTimeChanged();
            update();
            return;
        }
    } else if (isTokenDragging) {
        draggedTokenStartX = (mousePos.x() - draggedTokenDeltaX);
        draggedTokenStartY = mousePos.y() - TIME_BAR_HEIGHT;

        // Проверки и корректировки
        if (draggedTokenStartX < 0) draggedTokenStartX = 0;
        if (draggedTokenStartY < 0) draggedTokenStartY = 0;

        int maxStartX = (trackTactCount * tactDuration - draggedToken-
>relativeDurationView) * scaleFactor; //maxStartX в больших
        if (draggedTokenStartX > maxStartX)
            draggedTokenStartX = maxStartX;

        int maxStartY = (model->rowCount() - 1) * trackHeight;
        if (draggedTokenStartY > maxStartY)
            draggedTokenStartY = maxStartY;

        // Привязка к четвертям такта
        if (!(event->modifiers() & Qt::ShiftModifier)) {
            double quarterDuration = (tactDuration * scaleFactor / 4);
            draggedTokenStartX = round(draggedTokenStartX / quarterDuration) *
quarterDuration;
        }

        update();
    }

    if (isSlicing) {
        slicingX = mousePos.x() / scaleFactor;
    }
}

void AudioTrackFrame::mouseReleaseEvent(QMouseEvent *event) {
    if (event->button() == Qt::LeftButton) {
        if (isSlicing) {
            // Проверяем, был ли клик по токenu на аудиодорожке с индексом 0
            QPoint mousePos = event->pos();
            int yClick = mousePos.y() - TIME_BAR_HEIGHT;
            int trackIndex = yClick / trackHeight;

            if (trackIndex == 0) { // Только для дорожки с индексом 0
                AudioTrack *track = model->tracks[trackIndex];

                for (int tokenIndex = 0; tokenIndex < track->tokens.size();
++tokenIndex) {
                    AudioToken token = track->tokens[tokenIndex];
                    double x = token.startPositionView * scaleFactor;
                    double w = token.relativeDurationView * scaleFactor;

                    if (mousePos.x() > x && mousePos.x() < (x + w)) {
                        // Найден токен, по которому кликнули
                        double sliceX = slicingX;

                        if (sliceX > token.startPositionView && sliceX <
(token.startPositionView + token.relativeDurationView)) {
                            // Разрезаем токен
                            // Создаем два новых токена
                            AudioToken firstToken = token;
                            firstToken.relativeDurationView = sliceX -
token.startPositionView;

                            AudioToken secondToken = token;

```

```

        secondToken.startPositionView = sliceX;
        secondToken.relativeStartTimeView =
firstToken.relativeDurationView;
        secondToken.relativeDurationView =
token.relativeDurationView - firstToken.relativeDurationView;

        // Удаляем исходный токен и добавляем два новых
        track->tokens.erase(track->tokens.begin() + tokenIndex);
        track->tokens.insert(track->tokens.begin() + tokenIndex,
secondToken);
        track->tokens.insert(track->tokens.begin() + tokenIndex,
firstToken);

        // Сбрасываем флаг isSlicing
        isSlicing = false;

        // Обновляем отображение
        update();
        return;
    }
}

// Если токен не найден или клик не по дорожке с индексом 0
isSlicing = false;
}

if (isTokenDragging) {
    isTokenDragging = false;

    int dropX = draggedTokenStartX;
    if (!(event->modifiers() & Qt::ShiftModifier)) {
        double quarterDuration = tactDuration * scaleFactor / 4;
        dropX = round(draggedTokenStartX / quarterDuration) * quarterDuration;
// dropX в больших координатах
    }

    int dropTrackIndex = (draggedTokenStartY) / trackHeight;
    int oldTrackIndex = draggedToken->audioTrack;

    model->moveToken(oldTrackIndex, dropTrackIndex, draggedTokenIndex, dropX /
scaleFactor);
    emit trackChanged(oldTrackIndex);
    emit trackChanged(dropTrackIndex);

    delete draggedToken;
    draggedToken = nullptr;
} else if (isCurrTimeChanging) {
    isCurrTimeChanging = false;
}

update();
}
}

void AudioTrackFrame::showTrackContextMenu(const QPoint &pos, int trackIndex) {
    QMenu contextMenu(this);

    QAction *deleteAction = contextMenu.addAction("Delete audiotrack");
    connect(deleteAction, &QAction::triggered, this, [=]() {
        deleteTrack(trackIndex);
    });

    // Устанавливаем флаг выбранной дорожки
    AudioTrack *currTrack = model->tracks[trackIndex];
    currTrack->isSelected = true;

    // Сбрасываем флаг, когда меню закрывается
    connect(&contextMenu, &QMenu::aboutToHide, this, [=]() {

```

```

        currTrack->isSelected = false;
    });

    contextMenu.exec(mapToGlobal(pos));
}

void AudioTrackFrame::showTokenContextMenu(const QPoint &pos, int trackIndex, int
tokenIndex) {
    QMenu contextMenu(this);

    AudioToken *currToken = &model->tracks[trackIndex]->tokens[tokenIndex];
    currToken->setSelectedFlag();

    // Сбрасываем флаг, когда меню закрывается
    connect(&contextMenu, &QMenu::aboutToHide, this, [currToken]() {
        currToken->resetSelectedFlag();
    });

    QAction *deleteAction = contextMenu.addAction("Delete token");
    connect(deleteAction, &QAction::triggered, this, [=]() {
        deleteToken(trackIndex, tokenIndex);
    });

    QAction *editAction = contextMenu.addAction("Edit token");
    connect(editAction, &QAction::triggered, this, [=]() {
        openEditTokenWindow(trackIndex, tokenIndex);
    });

    contextMenu.exec(mapToGlobal(pos));
}

void AudioTrackFrame::deleteTrack(int trackIndex) {
    if (trackIndex < 0 || trackIndex >= model->rowCount()) return;

    delete model->tracks[trackIndex];
    model->tracks.removeAt(trackIndex);

    update(); // Перерисовать интерфейс
}

void AudioTrackFrame::deleteToken(int trackIndex, int tokenIndex) {
    if (trackIndex < 0 || trackIndex >= model->rowCount()) return;

    AudioTrack *track = model->tracks[trackIndex];
    if (tokenIndex < 0 || tokenIndex >= track->tokens.size()) return;

    track->tokens.removeAt(tokenIndex);
    track->isTrackChanged = true;

    update(); // Перерисовать интерфейс
}

void AudioTrackFrame::resizeToFitContent() {
    if (!model) return;

    // Рассчитываем ширину и высоту на основе количества тактов и дорожек
    int width = trackTactCount * tactDuration * scaleFactor;
    int height = model->rowCount() * trackHeight;

    // Устанавливаем минимальные размеры для trackFrame
    setMinimumWidth(width);
    setMinimumHeight(height);

    // Обновляем виджет
    updateGeometry();
    update();
}

void AudioTrackFrame::wheelEvent(QWheelEvent *event) {
    if (event->modifiers() & Qt::ControlModifier) {

```



```

        double factor = event->angleDelta().y() > 0 ? 1.1 : 0.9;
        scaleFactor *= factor;
        scaleFactor = qBound(MIN_SCALE, scaleFactor, MAX_SCALE); // Ограничиваем
масштаб
        resizeToFitContent();
        update();
    } else {
        QFrame::wheelEvent(event);
    }
}

//drag and drop из списка:
void AudioTrackFrame::dragEnterEvent(QDragEnterEvent *event) {
    if (event->mimeTypeData()->hasFormat("application/x-audiofile")) { // Проверяем MIME-
тип
        event->acceptProposedAction();
        isTokenDragging = true;

        QPoint mousePos = event->position().toPoint();
        int xClick = mousePos.x();
        int yClick = mousePos.y() - TIME_BAR_HEIGHT;

        int dropTrack = yClick / trackHeight;

        int uniqueAudiofileID = QString::fromUtf8(event->mimeTypeData()-
>data("application/x-audiofile")).toInt();

        draggedToken = new AudioToken(uniqueAudiofileID, xClick / scaleFactor,
AudioFileLinker::calculateDuration(uniqueAudiofileID), dropTrack, fileLinker);

        draggedTokenDeltaX = 0; //startPosition при 1.0
        draggedTokenDeltaY = trackHeight / 2;

        draggedTokenStartX = xClick - draggedTokenDeltaX; //startX и deltaX в
больших координатах
        draggedTokenStartY = yClick - draggedTokenDeltaY;
        draggedTokenIndex = -1;
    }
}

void AudioTrackFrame::dragMoveEvent(QDragMoveEvent *event) {
    if (event->mimeTypeData()->hasFormat("application/x-audiofile")) {
        QPoint mousePos = event->position().toPoint();
        draggedTokenStartX = mousePos.x();
        draggedTokenStartY = mousePos.y() - TIME_BAR_HEIGHT;

        // Ограничиваем перетаскиваемые координаты
        if (draggedTokenStartX < 0) draggedTokenStartX = 0;
        if (draggedTokenStartY < 0) draggedTokenStartY = 0;

        int maxStartX = trackTactCount * tactDuration * scaleFactor;
        if (draggedTokenStartX > maxStartX)
            draggedTokenStartX = maxStartX;

        int maxStartY = (model->rowCount() - 1) * trackHeight;
        if (draggedTokenStartY > maxStartY)
            draggedTokenStartY = maxStartY;

        if (!(event->modifiers() & Qt::ShiftModifier)) {
            double quarterDuration = (tactDuration * scaleFactor / 4);
            draggedTokenStartX = round(draggedTokenStartX / quarterDuration) *
quarterDuration;
        }

        update(); // Перерисовываем временный токен
        event->acceptProposedAction();
    }
}

void AudioTrackFrame::dropEvent(QDropEvent *event) {

```

```

        if (event->mimeType() != "application/x-audiofile") {
            int dropX = draggedTokenStartX;
            if (!(event->modifiers() & Qt::ShiftModifier)) {
                double quarterDuration = tactDuration * scaleFactor / 4;
                dropX = round(draggedTokenStartX / quarterDuration) * quarterDuration;
            }
            //dropX в больших координатах
            int dropTrack = (draggedTokenStartY) / trackHeight;

            int uniqueAudiofileID = QString::fromUtf8(event->mimeType() -
>data("application/x-audiofile")).toInt();

            // Привязка к четвертям такта
            if (!(event->modifiers() & Qt::ShiftModifier)) {
                double quarterDuration = tactDuration * scaleFactor / 4;
                dropX = round(dropX / quarterDuration) * quarterDuration;
            }

            AudioToken *newToken = new AudioToken(uniqueAudiofileID, dropX / scaleFactor,
AudioFileLinker::calculateDuration(uniqueAudiofileID), dropTrack, fileLinker);
            AudioTrack *track = model->tracks[dropTrack];
            track->addToken(*newToken);
            emit trackChanged(dropTrack);

            delete(draggedToken);
            draggedToken = nullptr;
            isTokenDragging = false;

            event->acceptProposedAction();
            update();
        }
    }

void AudioTrackFrame::onTrackAdded() {
    resizeToFitContent(); // Пересчитать размеры
    update();             // Перерисовать содержимое
}

bool AudioTrackFrame::hasSomeChanges() {
    for (AudioTrack *track: model->tracks) {
        if (track->isTrackChanged) return true;
    }
    return false;
}

void AudioTrackFrame::onPlayClicked() {
    AudioTrack *currTrack;
    if (hasSomeChanges()) {
        if (totalTrack->trackData) {
            delete (totalTrack->trackData);
            totalTrack->trackData = nullptr; //избегаем утечки памяти
        }

        QByteArray *res = new QByteArray();
        totalTrack = model->tracks[0];
        totalTrack->processAudioTrack();
        for (int i = 1; i < model->tracks.size(); i++) {
            currTrack = model->tracks[i];
            if (currTrack->tokens.isEmpty()) continue;

            if (currTrack->isTrackChanged)
                currTrack->processAudioTrack();

            *res = mixWavFiles(totalTrack->trackData, currTrack->trackData);
            totalTrack->trackData = res;
        }
    }
    currTime = currViewTime * MS_TO_PX;
    totalTrack->playTrack(currTime);
}

```

```

        connect(totalTrack, &AudioTrack::audioFileFinished, this,
&AudioTrackFrame::onAudioFileFinished);
        update();

        startTime = QDateTime::currentMSecsSinceEpoch(); // запоминаем время начала
        startViewTime = currViewTime;
        timer->start(1); // срабатывает каждые 1 мс
    }

void AudioTrackFrame::onPauseClicked() {
    if (timer && (timer->isActive())) {
        timer->stop();
        startViewTime = currViewTime;

        for (AudioTrack *track: model->tracks) {
            track->pauseTrack();
        }
        totalTrack->pauseTrack();
    }
    update();
}

void AudioTrackFrame::onStopClicked() {
    if (timer && (timer->isActive())) {
        timer->stop();
        startViewTime = currViewTime;

        for (AudioTrack *track: model->tracks) {
            track->pauseTrack();
        }
        totalTrack->pauseTrack();
    }
    update();
    currTime = 0;
    currViewTime = 0;
    startViewTime = 0;
}

void AudioTrackFrame::onExitClicked() {
    // Создание сообщения с подтверждением выхода
    QMessageBox msgBox(this);
    msgBox.setWindowTitle("Exit");
    msgBox.setIcon(QMessageBox::Question); // Установка иконки с вопросительным знаком
    msgBox.setText("Are you sure you want to quit?");
    msgBox.setStandardButtons(QMessageBox::Yes | QMessageBox::No);

    // Применение стиля к QMessageBox
    msgBox.setStyleSheet(
        "QMessageBox { background-color: " + ProjectConfiguration::clSidePanel.name()
+ "; color: " + ProjectConfiguration::clSidePanelText.name() + "; }"
        + "QPushButton { background-color: " +
ProjectConfiguration::clDarkButton.name() + "; color: " +
ProjectConfiguration::clSidePanelText.name() + "; }"
        + "QPushButton: hover { background-color: " +
ProjectConfiguration::clDarkButtonSelected.name() + "; }"
    );

    // Отображение сообщения и получение ответа
    QMessageBox::StandardButton reply =
static_cast<QMessageBox::StandardButton>(msgBox.exec());
    if (reply == QMessageBox::Yes) {
        QApplication::quit();
    }
}

void AudioTrackFrame::updateCurrTime() {
    int elapsedTime = QDateTime::currentMSecsSinceEpoch() - startTime; // прошедшее
время
    currTime += elapsedTime;
    currViewTime = round((startViewTime * MS_TO_PX + elapsedTime) / MS_TO_PX);
}

```

```

        update(); // перерисовываем содержимое
    }

void AudioTrackFrame::onVerticalScrollBarChanged() {
    update();
}

void AudioTrackFrame::onCurrTimeChanged() {
    startViewTime = currViewTime;
    currTime = currViewTime * MS_TO_PX;
    startTime = QDateTime::currentMSecsSinceEpoch();
}

void AudioTrackFrame::onTrackChanged(int index){
    model->tracks[index]->setTrackChangeFlag();
}

QByteArray AudioTrackFrame::mixWavFiles(QByteArray *wav1, QByteArray *wav2) {
    // Проверяем, что оба файла имеют минимум заголовок
    if (wav1->size() < 44 || wav2->size() < 44) {
        qWarning() << "Invalid WAV files: too small!";
        return QByteArray();
    }

    // Извлекаем заголовки
    QByteArray header1 = wav1->left(44);
    QByteArray header2 = wav2->left(44);

    // Проверяем, что параметры WAV-файлов совпадают
    if (header1.mid(22, 2) != header2.mid(22, 2) || // Количество каналов
        header1.mid(24, 4) != header2.mid(24, 4) || // Частота дискретизации
        header1.mid(34, 2) != header2.mid(34, 2)) { // Битность
        qWarning() << "WAV files have different formats!";
        return QByteArray();
    }

    // Извлекаем параметры WAV
    int16_t numChannels = qFromLittleEndian<int16_t>(reinterpret_cast<const uchar
*>(header1.data() + 22));
    int32_t sampleRate = qFromLittleEndian<int32_t>(reinterpret_cast<const uchar
*>(header1.data() + 24));
    int16_t bitsPerSample = qFromLittleEndian<int16_t>(reinterpret_cast<const uchar
*>(header1.data() + 34));
    int16_t bytesPerSample = bitsPerSample / 8;

    if (bitsPerSample != 16) {
        qWarning() << "Only 16-bit WAV files are supported!";
        return QByteArray();
    }

    // Извлекаем аудиоданные
    QByteArray data1 = wav1->mid(44);
    QByteArray data2 = wav2->mid(44);

    // Определяем длину аудиоданных
    int maxLength = qMax(data1.size(), data2.size());

    // Дополняем меньший файл нулями до длины большего
    if (data1.size() < maxLength) {
        data1.append(QByteArray(maxLength - data1.size(), 0));
    }
    if (data2.size() < maxLength) {
        data2.append(QByteArray(maxLength - data2.size(), 0));
    }

    // Смешиваем аудиоданные
    QByteArray mixedData;
    mixedData.resize(maxLength);

    for (int i = 0; i < maxLength; i += bytesPerSample) {

```

```

        // Берем сэмплы как 16-битные значения
        int16_t sample1 = qFromLittleEndian<int16_t>(reinterpret_cast<const uchar
*>(data1.data() + i));
        int16_t sample2 = qFromLittleEndian<int16_t>(reinterpret_cast<const uchar
*>(data2.data() + i));

        // Смешиваем сэмплы
        int32_t mixedSample = sample1 + sample2;

        // Ограничиваем диапазон значения
        if (mixedSample > INT16_MAX) mixedSample = INT16_MAX;
        if (mixedSample < INT16_MIN) mixedSample = INT16_MIN;

        // Добавляем смешанный сэмпл в результирующий массив
        qToLittleEndian<int16_t>(static_cast<int16_t>(mixedSample),
reinterpret_cast<uchar *>(mixedData.data() + i));
    }

    // Создаем новый заголовок WAV
    QByteArray newHeader = header1;
    int32_t newDataSize = mixedData.size();
    int32_t newFileSize = 36 + newDataSize;

    // Обновляем размер файла в заголовке
    qToLittleEndian<int32_t>(newFileSize, reinterpret_cast<uchar *>(newHeader.data() +
4));
    // Обновляем размер аудиоданных в заголовке
    qToLittleEndian<int32_t>(newDataSize, reinterpret_cast<uchar *>(newHeader.data() +
40));

    // Соединяем заголовок и данные
    QByteArray result = newHeader + mixedData;
    return result;
}

void AudioTrackFrame::onAudioFileFinished() {
    if (!totalTrack->tokens.isEmpty()) {
        timer->stop();
        currTime = 0;
        currViewTime = 0;
        startTime = QDateTime::currentMSecsSinceEpoch(); // запоминаем время начала
        startViewTime = currViewTime;
        timer->start(1); // срабатывает каждые 1 мс
        update();
    }
}

void AudioTrackFrame::openEditTokenWindow(int trackIndex, int tokenIndex) {
    AudioTrackDialog dialog(model, trackIndex, tokenIndex, fileLinker, this);
    if (dialog.exec() == QDialog::Accepted) {
        deleteToken(trackIndex, tokenIndex);
        for (AudioToken token : dialog.dialogTokens) {
            model->tracks[trackIndex]->addToken(token);
        }
        update();
    }
}

#include "audiotoken.h"

#include <QPainter>
#include "../projectconfiguration.h"
#include "../audiofilelinker.h"

int AudioToken::idCounter = 0;

AudioToken::AudioToken(int audiofileID, double startPosition, double duration, int
audioTrack,
                        AudioFileLinker *linker,

```

```

        QColor headerColor,
        QColor mainContentColor):

    audiofileID(audiofileID),
    startPositionView(startPosition),
    durationView(duration),
    audioTrack(audioTrack),
    relativeStartTimeView(0),
    relativeDurationView(duration),
    headerColor(headerColor),
    mainContentColor(mainContentColor),
    isSelected(false),
    linker(linker)
{
    tokenID = idCounter++;
}

AudioToken::AudioToken(int audiofileID, double startPosition, double duration, int
audioTrack,
                        AudioFileLinker *linker, double relStartPosition, double
relDuration,
                        QColor headerColor,
                        QColor mainContentColor):

    audiofileID(audiofileID),
    startPositionView(startPosition),
    durationView(duration),
    audioTrack(audioTrack),
    relativeStartTimeView(relStartPosition),
    relativeDurationView(relDuration),
    headerColor(headerColor),
    mainContentColor(mainContentColor),
    isSelected(false),
    linker(linker)
{
    tokenID = idCounter++;
}

void AudioToken::drawToken(QPainter *painter, double scaleFactor, const int
trackHeight) const {
    int x = this->startPositionView * scaleFactor; // Масштабируем позицию
    int w = this->relativeDurationView * scaleFactor; // Масштабируем ширину
    int y = TIME_BAR_HEIGHT + this->audioTrack * trackHeight;

    int headerHeight = TRACK_HEIGHT * TOKEN_HEADER_RELATIVE_HEIGHT;
    int mainContentHeight = trackHeight - headerHeight;

    // Верхняя часть токена с текстом
    painter->setPen(ProjectConfiguration::clTokenText);
    painter->setBrush(isSelected ? ProjectConfiguration::clSelectedAudioTokenHeader :
headerColor);
    painter->drawRect(x, y, w, headerHeight);
    painter->drawText(x + 5, y + headerHeight / 4 * 3, QString::number(audiofileID));

    // Нижняя часть для отображения аудиосигнала
    painter->setBrush(isSelected ?
ProjectConfiguration::clSelectedAudioTokenMainContent : mainContentColor);
    painter->drawRect(x, y + headerHeight, w, mainContentHeight);

    // Рисуем аналоговый сигнал
    if (linker) {
        const QByteArray &audioData = linker->audioFiles[audiofileID]->audioData;
        if (audioData.size() > 44) { // Убедимся, что есть данные после заголовка
            const int16_t *samples = reinterpret_cast<const
int16_t*>(audioData.constData() + 44); // Пропускаем 44 байта заголовка
            int sampleCount = (audioData.size() - 44) / sizeof(int16_t); // Количество
выборок
            double durationInMs = this->relativeDurationView * MS_TO_PX; //
Длительность токена в миллисекундах

```

```

        double startInMs = this->relativeStartTimeView * MS_TO_PX; // Начальное
        время токена в миллисекундах
        double samplesPerMs = sampleCount / (linker->audioFiles[audiofileID]-
        >durationMs); // Количество выборок на миллисекунду

        int startSample = static_cast<int>(startInMs * samplesPerMs); // Номер
        начальной выборки
        int sampleRange = static_cast<int>(durationInMs * samplesPerMs); //
        Количество выборок, охватываемых токеном

        if (startSample + sampleRange > sampleCount) {
            sampleRange = sampleCount - startSample; // Ограничиваем выборки,
            чтобы не выходить за пределы массива
        }

        // Пропорция между шириной токена и количеством выборок
        double samplesPerPixel = static_cast<double>(sampleRange) / w;

        // Рисуем сигнал
        painter->setPen(Qt::white); // Цвет линии сигнала
        int centerY = y + headerHeight + mainContentHeight / 2; // Центр нижней
        части токена

        int stepWidth = 3; // Увеличиваем ширину одной ступеньки (в пикселях)
        double amplitudeMultiplier = 1.5; // Коэффициент для увеличения высоты
        сигнала

        for (int i = 0; i < (w - 1); i += stepWidth) {
            int sampleIndex = startSample + static_cast<int>(i * samplesPerPixel);
            if (sampleIndex < startSample + sampleRange) {
                int16_t sampleValue = samples[sampleIndex]; // Значение выборки
                double normalizedAmplitude = static_cast<double>(sampleValue) /
                INT16_MAX; // Нормализуем амплитуду
                int amplitudeHeight = static_cast<int>(normalizedAmplitude *
                (mainContentHeight / 2) * amplitudeMultiplier); // Увеличиваем высоту сигнала

                // Рисуем полосу (увеличенную ступеньку)
                painter->drawRect(x + i, centerY - amplitudeHeight, stepWidth,
                amplitudeHeight * 2); // Полоса вместо линии
            }
        }
    }
}

void AudioToken::updateTokenRelativeStartPosition(int newRelativeStartTimeView) {
    // Убедитесь, что новое значение не превышает границы
    if (newRelativeStartTimeView >= 0 && newRelativeStartTimeView <
    (relativeStartTimeView + relativeDurationView)) {
        relativeStartTimeView = newRelativeStartTimeView;
    }
}

void AudioToken::updateTokenRelativeDuration(double newDuration) {
    // Убедитесь, что новая длительность больше нуля
    if (newDuration > 0 && (relativeStartTimeView + newDuration) <= (startPositionView
    + durationView)) {
        relativeDurationView = newDuration;
    }
}

void AudioToken::resetSelectedFlag() {
    isSelected = false;
}

void AudioToken::setSelectedFlag() {
    isSelected = true;
}

```

```

#include "audiotrack.h"
#include <QList>
#include <QByteArray>
#include <QMap>
#include <algorithm>
#include <cstring>
#include <QDebug>
#include "../audiofilelinker.h"
#include "../audiotrackframe.h"

AudioTrack::AudioTrack(QObject *parent) : QObject(parent)
{
    trackData = new QByteArray;
    isTrackChanged = false;
    player = new QMediaPlayer;
    audioOutput = new QAudioOutput;
    buffer = new QBuffer;
    isSelected = false;
    connect(player, &QMediaPlayer::mediaStatusChanged, this,
    &AudioTrack::onMediaStatusChanged);
}

void AudioTrack::addToken(AudioToken &token) {
    tokens.append(token);
}

QList<AudioToken>& AudioTrack::getTokens() {
    return tokens;
}

QByteArray createWavHeader(int dataSize) {
    QByteArray header(WAV_HEADER_SIZE, 0);
    char *headerData = header.data();

    // RIFF заголовок
    std::memcpy(headerData, "RIFF", 4);
    int chunkSize = dataSize + 36; // Общий размер файла - 8 байт
    std::memcpy(headerData + 4, &chunkSize, 4);
    std::memcpy(headerData + 8, "WAVE", 4);

    // fmt подзаголовок
    std::memcpy(headerData + 12, "fmt ", 4);
    int subchunk1Size = 16; // Размер подзаголовка (16 для PCM)
    std::memcpy(headerData + 16, &subchunk1Size, 4);
    short audioFormat = 1; // PCM формат
    std::memcpy(headerData + 20, &audioFormat, 2);
    std::memcpy(headerData + 22, &CHANNELS, 2);
    std::memcpy(headerData + 24, &SAMPLE_RATE, 4);
    int byteRate = SAMPLE_RATE * BLOCK_ALIGN;
    std::memcpy(headerData + 28, &byteRate, 4);
    std::memcpy(headerData + 32, &BLOCK_ALIGN, 2);
    std::memcpy(headerData + 34, &BITS_PER_SAMPLE, 2);

    // data подзаголовок
    std::memcpy(headerData + 36, "data", 4);
    std::memcpy(headerData + 40, &dataSize, 4);

    return header;
}

// Функция для обработки аудиодорожки
void AudioTrack::processAudioTrack() {
    // 1. Сортировка токенов по возрастанию startPositionView
    std::sort(tokens.begin(), tokens.end(),
    [](const AudioToken &a, const AudioToken &b) {
        return a.startPositionView < b.startPositionView;
    });

    // 2. Объявляем результирующий массив для данных
    QByteArray *combinedAudioData = new QByteArray();
}

```



```

// Изначально заполняем массив заголовком WAV
combinedAudioData->append(createWavHeader(0)); // Заголовок обновим позже

// Переменная для отслеживания текущей позиции в байтах
int currentBytePos = 0;

for (const AudioToken &token : tokens) {
    // Получаем аудиоданные из карты файлов
    QByteArray audioFileData = AudioFileLinker::audioFiles[token.audiofileID]-
>audioData;

    QByteArray audioData = audioFileData.mid(WAV_HEADER_SIZE);

    // Рассчитываем смещение и длину активной части в байтах
    int startByte = static_cast<int>(token.relativeStartTimeView * MS_TO_PX *
MS_TO_BYTES);
    int durationBytes = static_cast<int>(token.relativeDurationView * MS_TO_PX *
MS_TO_BYTES);

    // Извлекаем активную часть аудиоданных токена
    QByteArray tokenAudioData = audioData.mid(startByte, durationBytes);

    // Рассчитываем позицию начала токена в результирующем массиве
    int tokenStartPos = static_cast<int>(token.startPositionView * MS_TO_PX *
MS_TO_BYTES);

    // Если между текущей позицией и началом токена есть промежуток, заполняем его
тишиной
    if (currentBytePos < tokenStartPos) {
        int silenceBytes = tokenStartPos - currentBytePos;
        silenceBytes += (BLOCK_ALIGN - (silenceBytes % BLOCK_ALIGN)) %
BLOCK_ALIGN; // Дополняем до кратного BLOCK_ALIGN
        combinedAudioData->append(QByteArray(silenceBytes, 0));
        currentBytePos = tokenStartPos;
    }

    // Проверяем кратность размера данных BLOCK_ALIGN
    if (tokenAudioData.size() % BLOCK_ALIGN != 0) {
        int padding = BLOCK_ALIGN - (tokenAudioData.size() % BLOCK_ALIGN);
        tokenAudioData.append(QByteArray(padding, 0)); // Дополняем до кратного
BLOCK_ALIGN
    }

    // Добавляем данные токена в общий массив
    combinedAudioData->append(tokenAudioData);
    currentBytePos += tokenAudioData.size();
}

// Обновляем размер data в заголовке WAV
int dataSize = combinedAudioData->size() - WAV_HEADER_SIZE;
QByteArray header = createWavHeader(dataSize);
std::memcpy(combinedAudioData->data(), header.data(), WAV_HEADER_SIZE);

trackData = combinedAudioData;
isTrackChanged = false;
}

void AudioTrack::playTrack(qint64 startTime) {
    player->setAudioOutput(audioOutput);
    audioOutput->setVolume(80);
    if (buffer->isOpen())
        buffer->close();
    player->setSourceDevice(nullptr);
    buffer->setData(*trackData);

    buffer->open(QIODevice::ReadOnly);
    player->setSourceDevice(buffer);

    player->play();
}

```

```

        // Подключение к сигналу stateChanged
        connect(player, &QMediaPlayer::playbackStateChanged, this, [this,
startTime](QMediaPlayer::PlaybackState state) {
            if (state == QMediaPlayer::PlayingState) {
                player->setPosition(startTime);
            }
        });

        connect(player, &QMediaPlayer::mediaStatusChanged, this, [this,
startTime](QMediaPlayer::MediaStatus status) {
            if (status == QMediaPlayer::EndOfMedia) {
                player->pause();

                connect(player, &QMediaPlayer::playbackStateChanged, this, [this,
startTime](QMediaPlayer::PlaybackState state) {
                    if (state == QMediaPlayer::PlayingState) {
                        player->setPosition(0);
                    }
                });

                emit audioFileFinished();
                player->play();
            }
        });
    }

void AudioTrack::pauseTrack(){
    if (player->isPlaying())
        player->stop();
}

void AudioTrack::onMediaStatusChanged(QMediaPlayer::MediaStatus status) {
    if (status == QMediaPlayer::EndOfMedia) {
        //buffer->close();
        //player->deleteLater();
        //audioOutput->deleteLater();
        //buffer->deleteLater(); // Освобождаем QBuffer
    }
}

void AudioTrack::setTrackChangeFlag(){
    isTrackChanged = true;
}

void AudioTrack::clearData() {
    QByteArray emptyHeader = createWavHeader(0);
    *trackData = emptyHeader;
}

#include "workspacemodel.h"

WorkspaceModel::WorkspaceModel(QObject *parent)
    : QAbstractListModel(parent) {}

int WorkspaceModel::rowCount(const QModelIndex &parent) const {
    Q_UNUSED(parent);
    return tracks.size();
}

QVariant WorkspaceModel::data(const QModelIndex &index, int role) const {
    if (!index.isValid() || index.row() < 0 || index.row() >= tracks.size())
        return QVariant();

    if (role == Qt::DisplayRole) {
        return QVariant("Audio Track " + QString::number(index.row()));
    }

    return QVariant();
}

```

```

}

void WorkspaceModel::addTrack(AudioTrack *track) {
    beginInsertRows(QModelIndex(), tracks.size(), tracks.size());
    tracks.append(track);
    endInsertRows();
}

QList<AudioTrack*> WorkspaceModel::getTracks() {
    return tracks;
}

void WorkspaceModel::moveToken(int oldTrackIndex, int newTrackIndex, int tokenIndex,
double newStartPosition) {
    if (oldTrackIndex < 0 || oldTrackIndex >= tracks.size() ||
        newTrackIndex < 0 || newTrackIndex >= tracks.size()) {
        return; // Неверные индексы
    }

    // Извлекаем токен из старой дорожки
    AudioToken token = tracks[oldTrackIndex]->tokens.takeAt(tokenIndex);

    // Обновляем поля токена
    token.audioTrack = newTrackIndex;
    token.startPositionView = newStartPosition;

    // Добавляем токен в новую дорожку
    tracks[newTrackIndex]->addToken(token);

    // Уведомляем об изменениях
    emit dataChanged(index(oldTrackIndex), index(oldTrackIndex));
    emit dataChanged(index(newTrackIndex), index(newTrackIndex));
}

void WorkspaceModel::clear() {
    beginResetModel();
    for (AudioTrack* track : tracks) {
        track->pauseTrack();
    }
    qDeleteAll(tracks);
    tracks.clear();
    endResetModel();
}

#include "audiofilelinker.h"
#include "audiofileobject.h"
#include "projectconfiguration.h"

QList<AudioFileObject*> AudioFileLinker::audioFiles; // Инициализация статического
поля

AudioFileLinker::AudioFileLinker() {}

int AudioFileLinker::appendAudioFile(const QString &filePath) {
    AudioFileObject *newItem = new AudioFileObject(filePath);
    audioFiles.append(newItem);
    return getLastID();
}

int AudioFileLinker::getLastID() const {
    return audioFiles.size() - 1;
}

QString AudioFileLinker::getFilePathByID(int ID) const {
    if (ID >= 0 && ID < audioFiles.size()) {
        AudioFileObject *item = audioFiles.at(ID);
        if (item && !item->isEmpty() && item->canAccess()) {
            return item->audioFilePath;
        }
    }
}

```

```

    }
    return "";
}

double AudioFileLinker::calculateDuration(int ID) {
    if (ID >= 0 && ID < audioFiles.size()) {
        AudioFileObject *item = audioFiles.at(ID);
        return item->durationMs / MS_TO_PX;
    }
    return 0.0; // Возвращаем 0, если ID невалиден
}

#include "audiofilelist.h"
#include "projectconfiguration.h"
#include "audiofilelinker.h"

#include <QHeaderView>
#include <QFileInfo>
#include <QMimeData>
#include <QDrag>
#include <QStandardItemModel>
#include <QDir>

AudioFileList::AudioFileList(QString path, QWidget *parent, AudioFileLinker *linker) :
QListView(parent) {
    model = new QStandardItemModel(this);
    setModel(model);
    setDragEnabled(true);
    setAcceptDrops(true);
    setDropIndicatorShown(true);

    // Установка стилей для выделения объектов
    setStyleSheet("QListView { background-color: " +
ProjectConfiguration::clSidePanel.name() + "; color: " +
ProjectConfiguration::clSidePanelText.name() + "; }"
        + "QListView::item { height: " +
QString::number(DEFAULT_LIST_ITEM_HEIGHT) + "px; }" // Увеличение высоты элементов
        + "QListView::item:selected { background-color: " +
ProjectConfiguration::clSidePanelItemSelected.name() + "; }"
        + "QScrollBar:vertical { border: none; background: " +
ProjectConfiguration::clScrollbarBackground.name() + "; width: " +
QString::number(DEFAULT_SCROLLBAR_SIZE) + "px; }"
        + "QScrollBar::handle:vertical { background: " +
ProjectConfiguration::clScrollbar.name() + "; }"
        + "QScrollBar:horizontal { border: none; background: " +
ProjectConfiguration::clScrollbarBackground.name() + "; height: " +
QString::number(DEFAULT_SCROLLBAR_SIZE) + "px; }"
        + "QScrollBar::handle:horizontal { background: " +
ProjectConfiguration::clScrollbar.name() + "; }"
        + "QMenu { background-color: " +
ProjectConfiguration::clSidePanel.name() + "; color: " +
ProjectConfiguration::clSidePanelText.name() + "; }"
        + "QMenu::item:selected { background-color: " +
ProjectConfiguration::clMenuBarItemSelected.name() + "; }"
    );

    audioFileLinker = linker;
    dirPath = path;
    if (path != "")
        loadAudioFiles(path);
}

void AudioFileList::loadAudioFiles(const QString &directoryPath) {
    QDir dir(directoryPath);
    QStringList filters;
    filters << "*.mp3" << "*.wav" << "*.ogg";
    QFileInfoList fileInfoList = dir.entryInfoList(filters, QDir::Files);

    model->clear(); // Очистка модели перед загрузкой новых файлов
}

```

```

        AudioFileLinker::audioFiles.clear();

        for (const QFileInfo &fileInfo : fileInfoList) {
            QStandardItem *item = new QStandardItem(QIcon::fromTheme(MP3_ICON_PATH),
fileInfo.fileName());

            audioFileLinker->appendAudioFile(fileInfo.absoluteFilePath());
            int currID = audioFileLinker->getLastID();
            item->setData(currID, Qt::UserRole);

            model->appendRow(item);
        }
    }

void AudioFileList::startDrag(Qt::DropActions supportedActions) {
    QModelIndex index = currentIndex();
    if (index.isValid()) {
        QMimeData *mimeData = new QMimeData;
        // Получаем уникальный ID из данных модели
        int uniqueID = index.data(Qt::UserRole).toInt();
        // Устанавливаем ID в MimeData
        mimeData->setData("application/x-audiofile", QByteArray::number(uniqueID));
        // Создаем QDrag и устанавливаем MimeData
        QDrag *drag = new QDrag(this);
        drag->setMimeData(mimeData);
        drag->setPixmap(QPixmap(MP3_ICON_PATH)); // Добавляем иконку для визуализации
        drag->exec(supportedActions);
    }
}

void AudioFileList::dragEnterEvent(QDragEnterEvent *event) {
    if (event->mimeData()->hasFormat("application/x-audiofile")) {
        event->acceptProposedAction();
    }
}

void AudioFileList::dragMoveEvent(QDragMoveEvent *event) {
    if (event->mimeData()->hasFormat("application/x-audiofile")) {
        event->acceptProposedAction();
    }
}

void AudioFileList::dropEvent(QDropEvent *event) {
    if (event->mimeData()->hasFormat("application/x-audiofile")) {
        QModelIndex targetIndex = indexAt(event->position().toPoint());
        QModelIndex sourceIndex = currentIndex();

        if (sourceIndex.isValid()) {
            if (model) {
                QStandardItem *item = model->takeItem(sourceIndex.row());
                if (item) {
                    // Вставляем элемент на новую позицию
                    if (targetIndex.isValid()) {
                        model->insertRow(targetIndex.row(), item);
                    } else {
                        model->appendRow(item);
                    }
                }
            }
        }
        removeEmptyRows();
        event->acceptProposedAction();
    }
}

void AudioFileList::removeEmptyRows() {
    if (model) {
        for (int row = model->rowCount() - 1; row >= 0; --row) {
            if (model->item(row) == nullptr) {
                model->removeRow(row);
            }
        }
    }
}

```

```

    }
}

#include "AudioTrackDialog.h"

// Реализация методов AudioTrackDialog
AudioTrackDialog::AudioTrackDialog(WorkspaceModel *baseModel, int trackIndex, int
tokenIndex, AudioFileLinker *linker, QWidget *parent)
    : QDialog(parent)
{
    setWindowTitle("Edit Audio Track");
    resize(800, 600);
    setWindowFlags(windowFlags() & ~Qt::WindowCloseButtonHint &
~Qt::WindowMinimizeButtonHint);

    QVBoxLayout *layout = new QVBoxLayout(this);

    this->baseModel = baseModel;
    editingToken = &(baseModel->tracks[trackIndex]->tokens[tokenIndex]);

    // Верхнее меню
    QMenuBar *menuBar = new QMenuBar(this);
    menuBar->setStyleSheet(
        "QMenuBar { background-color: " + ProjectConfiguration::clSidePanel.name() +
"; color: " + ProjectConfiguration::clSidePanelText.name() + "; }" +
        "QMenuBar::item:selected { background-color: " +
ProjectConfiguration::clMenuBarItemSelected.name() + "; }" +
        "QMenuBar::item:pressed { background-color: " +
ProjectConfiguration::clMenuBarItemSelected.name() + "; }" +
        "QMenu { background-color: " + ProjectConfiguration::clSidePanel.name() + ";
color: " + ProjectConfiguration::clSidePanelText.name() + "; }" +
        "QMenu::item:selected { background-color: " +
ProjectConfiguration::clMenuBarItemSelected.name() + "; }"
    );

    layout->setMenuBar(menuBar);

    // Центральная область с прокруткой
    QScrollArea *scrollArea = new QScrollArea(this);
    dialogTrackFrame = new AudioTrackFrame(DIALOG_TRACK_HEIGHT, this, scrollArea,
linker);
    dialogTrackFrame->setMinimumWidth(1000);
    dialogTrackFrame->setMinimumHeight(DIALOG_TRACK_HEIGHT + TIME_BAR_HEIGHT);

    scrollArea->setWidget(dialogTrackFrame);
    scrollArea->setWidgetResizable(true); // Позволяет динамически изменять размеры
scrollArea->setHorizontalScrollBarPolicy(Qt::ScrollBarAlwaysOn);

    WorkspaceModel *dialogModel = new WorkspaceModel();

    dialogTrack = new AudioTrack();
    dialogModel->addTrack(dialogTrack);
    dialogTrackFrame->setModel(dialogModel);
    dialogTrackFrame->show();

    AudioToken editingTokenCopy = baseModel->tracks[trackIndex]->tokens[tokenIndex];
    editingTokenCopy.startPositionView = 0;
    editingTokenCopy.audioTrack = 0;
    dialogTrack->addToken(editingTokenCopy);
    dialogTrack->isTrackChanged = true;

    layout->addWidget(scrollArea);

    // Нижние кнопки
    QHBoxLayout *buttonLayout = new QHBoxLayout();
    QPushButton *playButton = new QPushButton("Play", this);
    QPushButton *pauseButton = new QPushButton("Pause", this);
    QPushButton *stopButton = new QPushButton("Stop", this);
    QPushButton *sliceButton = new QPushButton("Slice", this);

```

```

QPushButton *confirmButton = new QPushButton("Confirm", this);
QPushButton *cancelButton = new QPushButton("Cancel", this);

buttonLayout->addWidget(playButton);
buttonLayout->addWidget(pauseButton);
buttonLayout->addWidget(stopButton);
buttonLayout->addWidget(sliceButton);
buttonLayout->addWidget(confirmButton);
buttonLayout->addWidget(cancelButton);

// Устанавливаем общий стиль для всех кнопок
setStyleSheet("QPushButton { background-color: " +
ProjectConfiguration::clDarkButton.name() + "; color: " +
ProjectConfiguration::clSidePanelText.name() + "; }"
+ "QPushButton:hover { background-color: " +
ProjectConfiguration::clDarkButtonSelected.name() + "; }");

layout->addLayout(buttonLayout);

// Ваша существующая часть кода
QMenu *fileMenu = menuBar->addMenu("File");

// Действие для экспорта
QAction *exportAction = fileMenu->addAction("Export");
connect(exportAction, &QAction::triggered, this, [this]() {
// Открываем диалоговое окно для выбора места сохранения
QString fileName = QFileDialog::getSaveFileName(this, "Save WAV File",
"wavfile.wav", "WAV Files (*.wav)");

if (!fileName.isEmpty()) {
QFile file(fileName);
if (file.open(QIODevice::WriteOnly)) {
dialogTrack->processAudioTrack();

if (dialogTrack->trackData == nullptr || dialogTrack->trackData-
>isEmpty()) {
QMessageBox::warning(this, "Error", "No data to write.");
return;
}

QByteArray data = *(dialogTrack->trackData);
qint64 bytesWritten = file.write(data);
if (bytesWritten == -1) {
QMessageBox::warning(this, "Error", "Failed to write data to
file.");
}
file.close();
} else {
QMessageBox::warning(this, "Error", "Could not open file for
writing.");
}
}
});

// Действие для Help
QAction *helpAction = menuBar->addAction("Help");
connect(helpAction, &QAction::triggered, this, [this]() {
// Создаем диалоговое окно
QDialog *helpDialog = new QDialog(this);
helpDialog->setWindowTitle("Help");

// Создаем QLabel с текстом
QLabel *helpLabel = new QLabel(
"РЕДАКТОР СЭМПЛА:\n\n"
"Данное окно предназначено для редактирования сэмпла.\n"
"Вы можете:\n"
"1. Разделить данный сэмпл с помощью инструмента Slice.\n"
"2. Вырезать часть сэмпла с помощью инструмента Slice, а затем удаления
ненужной части с помощью ПКМ + Delete token\n"
"3. Прослушать трек перед экспортом.\n"

```

```

        "4. Добавить измененный сэмпл на аудиодорожку, подтвердив изменение
        кнопкой Confirm.\n"
        "5. Экспортировать сэмпл в формате wav с помощью пункта меню Export.\n",
        helpDialog
    );

    // Настройка стиля
    QString styleSheet = "QDialog, QLabel { background-color: " +
ProjectConfiguration::clSidePanel.name() + "; color: " +
ProjectConfiguration::clSidePanelText.name() + "; }"
        + "QLabel {font-size: 12pt; font-family: Arial; }";

    helpDialog->setStyleSheet(styleSheet);
    helpLabel->setStyleSheet("border: none;"); // Убираем рамку у QLabel

    // Создаем кнопку для закрытия диалога
    QPushButton *closeButton = new QPushButton("Close", helpDialog);
    connect(closeButton, &QPushButton::clicked, helpDialog, &QDialog::accept);

    // Устанавливаем вертикальный layout
    QVBoxLayout *layout = new QVBoxLayout(helpDialog);
    layout->addWidget(helpLabel);
    layout->addWidget(closeButton);

    // Устанавливаем layout для диалога
    helpDialog->setLayout(layout);

    // Показываем диалог
    helpDialog->exec();
});

// Подключаем сигнал подтверждения
connect(confirmButton, &QPushButton::clicked, this,
&AudioTrackDialog::confirmChanges);
connect(cancelButton, &QPushButton::clicked, this,
&AudioTrackDialog::cancelChanges);
connect(sliceButton, &QPushButton::clicked, this,
&AudioTrackDialog::onSliceButtonClicked);

connect(pauseButton, &QPushButton::clicked, dialogTrackFrame,
&AudioTrackFrame::onPauseClicked);
connect(playButton, &QPushButton::clicked, dialogTrackFrame,
&AudioTrackFrame::onPlayClicked);
connect(stopButton, &QPushButton::clicked, dialogTrackFrame,
&AudioTrackFrame::onStopClicked);
}

void AudioTrackDialog::confirmChanges() {
    for (AudioToken &token : dialogTrack->tokens) {
        token.startPositionView += editingToken->startPositionView;
        token.audioTrack = editingToken->audioTrack;
    }
    dialogTokens = dialogTrack->tokens;
    dialogTrackFrame->isSlicing = false;
    dialogTrack->pauseTrack();
    accept();
}

void AudioTrackDialog::cancelChanges() {
    dialogTrackFrame->isSlicing = false;
    dialogTrack->pauseTrack();
    reject();
}

void AudioTrackDialog::onSliceButtonClicked() {
    dialogTrackFrame->isSlicing = !(dialogTrackFrame->isSlicing);
    dialogTrackFrame->setFocus();
}

#include "controlpanel.h"

```



```

#include "projectconfiguration.h"
#include "audiotrackframe.h"
#include "workspace-implementation/workspacemodel.h"

ControlPanel::ControlPanel(AudioTrackFrame *trackframe, QWidget *parent):
QFrame(parent) {
    playButton = new QPushButton("Play", this);
    pauseButton = new QPushButton("Pause", this);
    stopButton = new QPushButton("Stop", this);
    exitButton = new QPushButton("Exit", this);
    addTrackButton = new QPushButton("Add track", this);
    this->trackFrame = trackframe;

    QHBoxLayout *layout = new QHBoxLayout(this);
    layout->addWidget(addTrackButton);
    layout->addWidget(playButton);
    layout->addWidget(pauseButton);
    layout->addWidget(stopButton);
    layout->addWidget(exitButton);

    addTrackButton->setDisabled(true);
    playButton->setDisabled(true);
    pauseButton->setDisabled(true);
    stopButton->setDisabled(true);
    exitButton->setDisabled(true);

    // Устанавливаем общий стиль для всех кнопок
    setStyleSheet("QPushButton { background-color: " +
ProjectConfiguration::clDarkButton.name() + "; color: " +
ProjectConfiguration::clSidePanelText.name() + "; }"
        + "QPushButton:hover { background-color: " +
ProjectConfiguration::clDarkButtonSelected.name() + "; }"
        + "ControlPanel { background-color: " +
ProjectConfiguration::clSidePanel.name() + "; }");

    connect(addTrackButton, &QPushButton::clicked, this,
&ControlPanel::onAddTrackButtonClicked);
    connect(playButton, &QPushButton::clicked, this,
&ControlPanel::onPlayButtonClicked);
    connect(pauseButton, &QPushButton::clicked, this,
&ControlPanel::onPauseButtonClicked);
    connect(stopButton, &QPushButton::clicked, this,
&ControlPanel::onStopButtonClicked);
    connect(exitButton, &QPushButton::clicked, this,
&ControlPanel::onExitButtonClicked);
}

void ControlPanel::onAddTrackButtonClicked() {
    // Create a new track and add it to the model
    AudioTrack *newTrack = new AudioTrack();
    trackFrame->model->addTrack(newTrack);

    emit trackAdded();
}

void ControlPanel::onPlayButtonClicked() {
    emit playClicked();
}

void ControlPanel::onPauseButtonClicked() {
    emit pauseClicked();
}

void ControlPanel::onStopButtonClicked() {
    emit stopClicked();
}

```

```

void ControlPanel::onExitButtonClicked() {

    emit exitClicked();
}

void ControlPanel::onProjectCreated() {
    addTrackButton->setDisabled(false);
    playButton->setDisabled(false);
    pauseButton->setDisabled(false);
    stopButton->setDisabled(false);
    exitButton->setDisabled(false);
}

#include "mainwindow.h"
#include "audiofilelist.h"
#include "audiotrackframe.h"
#include "controlpanel.h"
#include "projectconfiguration.h"
#include <QApplication>
#include <QSpinBox>
#include <QMessageBox>
#include <QFileDialog>
#include <QLabel>
#include <QDialog>
#include <shlobj.h>
#include <commdlg.h>

MainWindow::MainWindow(QWidget *parent): QMainWindow(parent) {
    linker = new AudioFileLinker();
    scrollArea = new QScrollArea(this);
    trackFrame = new AudioTrackFrame(TRACK_HEIGHT, this, scrollArea, linker);
    controlPanel = new ControlPanel(trackFrame, this);
    defaultModel = new WorkspaceModel();
    projectPath = "";
    projectFullPath = "";
    audioList = new AudioFileList("", this, linker);
    isProjectCreated = false;

    setupUI();

    connect(this, &MainWindow::projectCreated, controlPanel,
    &ControlPanel::onProjectCreated);
    connect(this, &MainWindow::projectCreated, this, &MainWindow::onProjectCreated);
}

MainWindow::~MainWindow() {}

QString winOpenFile(const wchar_t* filter)
{
    OPENFILENAMEW ofn;
    wchar_t szFile[MAX_PATH] = {0};

    ZeroMemory(&ofn, sizeof(ofn));
    ofn.lStructSize = sizeof(ofn);
    ofn.hwndOwner = nullptr;
    ofn.lpstrFile = szFile;
    ofn.nMaxFile = MAX_PATH;
    ofn.lpstrFilter = filter; // L"WAV Files\0*.wav\0All\0*.*\0"
    ofn.nFilterIndex = 1;
    ofn.Flags = OFN_PATHMUSTEXIST | OFN_FILEMUSTEXIST;

    if (GetOpenFileNameW(&ofn))
        return QString::fromWCharArray(ofn.lpstrFile);

    return "";
}

QString winSaveFile(const wchar_t* filter, const wchar_t* defaultExt)
{

```

```

OPENFILENAMEW ofn;
wchar_t szFile[MAX_PATH] = {0};

ZeroMemory(&ofn, sizeof(ofn));
ofn.lStructSize = sizeof(ofn);
ofn.hwndOwner = nullptr;
ofn.lpstrFile = szFile;
ofn.nMaxFile = MAX_PATH;
ofn.lpstrFilter = filter;
ofn.lpstrDefExt = defaultExt;
ofn.Flags = OFN_PATHMUSTEXIST | OFN_OVERWRITEPROMPT;

if (GetSaveFileNameW(&ofn))
    return QString::fromWCharArray(ofn.lpstrFile);

return "";
}

QString winSelectFolder()
{
    BROWSEINFOW bi = {0};
    bi.lpszTitle = L"Select Project Directory";

    LPITEMIDLIST pidl = SHBrowseForFolderW(&bi);
    wchar_t path[MAX_PATH];

    if (pidl != 0)
    {
        if (SHGetPathFromIDListW(pidl, path))
            return QString::fromWCharArray(path);
    }

    return "";
}

void msgInfo(const wchar_t* text, const wchar_t* title = L"Info")
{
    MessageBoxW(nullptr, text, title, MB_OK | MB_ICONINFORMATION);
}

void msgWarning(const wchar_t* text, const wchar_t* title = L"Warning")
{
    MessageBoxW(nullptr, text, title, MB_OK | MB_ICONWARNING);
}

int msgYesNo(const wchar_t* text, const wchar_t* title = L"Confirm")
{
    return MessageBoxW(nullptr, text, title, MB_YESNO | MB_ICONQUESTION);
}

void MainWindow::setupUI() {
    QWidget *centralWidget = new QWidget(this);
    setCentralWidget(centralWidget);
    setStyleSheet("background-color: " + ProjectConfiguration::clBackground.name() +
"; color: " + ProjectConfiguration::clSidePanelText.name() + ";");
    this->setMinimumSize(MIN_MAIN_WINDOW_WIDTH, MIN_MAIN_WINDOW_HEIGHT);

    QHBoxLayout *mainLayout = new QHBoxLayout(centralWidget);
    QVBoxLayout *sidePanelLayout = new QVBoxLayout();
    QVBoxLayout *workspaceLayout = new QVBoxLayout();

    mainLayout->addLayout(sidePanelLayout);
    mainLayout->addLayout(workspaceLayout);

    audioList->setFixedWidth(this->width() * DEFAULT_AUDIO_LIST_RELATIVE_WIDTH);
    sidePanelLayout->addWidget(audioList);

    scrollArea->setStyleSheet(

```

```

        "QFrame { background-color: " + ProjectConfiguration::clSidePanel.name() + ";
color: " + ProjectConfiguration::clSidePanelText.name() + "; }"
        + "QScrollBar:vertical { border: none; background: " +
ProjectConfiguration::clScrollbarBackground.name() + "; width: " +
QString::number(DEFAULT_SCROLLBAR_SIZE) + "px; }"
        + "QScrollBar::handle:vertical { background: " +
ProjectConfiguration::clScrollbar.name() + ";}"
        + "QScrollBar:horizontal { border: none; background: " +
ProjectConfiguration::clScrollbarBackground.name() + "; height: " +
QString::number(DEFAULT_SCROLLBAR_SIZE) + "px; }"
        + "QScrollBar::handle:horizontal { background: " +
ProjectConfiguration::clScrollbar.name() + ";}"
        + "QMenu { background-color: " + ProjectConfiguration::clSidePanel.name() + ";
color: " + ProjectConfiguration::clSidePanelText.name() + "; }"
        + "QMenu::item:selected { background-color: " +
ProjectConfiguration::clMenuBarItemSelected.name() + "; }"
    );

    trackFrame->setMinimumWidth(1000);
    trackFrame->setMinimumHeight(400);

    scrollArea->setWidget(trackFrame);
    scrollArea->setWidgetResizable(true);

    workspaceLayout->addWidget(scrollArea);
    trackFrame->setModel(defaultModel);
    trackFrame->show();

    controlPanel->setFixedHeight(this->height() *
DEFAULT_CONTROL_PANEL_RELATIVE_HEIGHT);
    workspaceLayout->addWidget(controlPanel);

    QMenuBar *menuBar = this->menuBar();
    QMenu *fileMenu = menuBar->addMenu("File");

    QAction *createProjectAction = fileMenu->addAction("Create project");
    QAction *openProjectAction = fileMenu->addAction("Open project");
    QAction *saveProjectAction = fileMenu->addAction("Save project");
    fileMenu->addSeparator();
    QAction *importAudioAction = fileMenu->addAction("Import audiofiles");
    QAction *exportAudioAction = fileMenu->addAction("Export audiofile");
    fileMenu->addSeparator();
    QAction *closeAction = fileMenu->addAction("Close");

    QAction *aboutAction = menuBar->addAction("About");

    QAction *helpAction = menuBar->addAction("Help");

    menuBar->setStyleSheet(
        "QMenuBar { background-color: " + ProjectConfiguration::clSidePanel.name() +
"; color: " + ProjectConfiguration::clSidePanelText.name() + "; }" +
        "QMenuBar::item:selected { background-color: " +
ProjectConfiguration::clMenuBarItemSelected.name() + "; }" +
        "QMenuBar::item:pressed { background-color: " +
ProjectConfiguration::clMenuBarItemSelected.name() + "; }" +
        "QMenu { background-color: " + ProjectConfiguration::clSidePanel.name() + ";
color: " + ProjectConfiguration::clSidePanelText.name() + "; }" +
        "QMenu::item:selected { background-color: " +
ProjectConfiguration::clMenuBarItemSelected.name() + "; }"
    );

    this->setWindowState(Qt::WindowMaximized);

    trackFrame->setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Expanding);
    audioList->setSizePolicy(QSizePolicy::Fixed, QSizePolicy::Expanding);

    connect(controlPanel, &ControlPanel::trackAdded, trackFrame,
&AudioTrackFrame::onTrackAdded);
    connect(controlPanel, &ControlPanel::playClicked, trackFrame,
&AudioTrackFrame::onPlayClicked);

```

```

        connect(controlPanel, &ControlPanel::pauseClicked, trackFrame,
&AudioTrackFrame::onPauseClicked);
        connect(controlPanel, &ControlPanel::stopClicked, trackFrame,
&AudioTrackFrame::onStopClicked);
        connect(controlPanel, &ControlPanel::exitClicked, trackFrame,
&AudioTrackFrame::onExitClicked);

// Реализация пунктов меню
connect(createProjectAction, &QAction::triggered, this, [&]() {
    QString dir = winSelectFolder();
    if (dir.isEmpty()) return;

    projectPath = dir;
    QDir directory(dir);
    projectDir = directory;

    directory.mkdir("resources");

    projectFullPath = directory.filePath(DEFAULT_PROJECT_NAME);
    QFile projectFile(projectFullPath);
    if (projectFile.open(QIODevice::WriteOnly))
        projectFile.close();

    defaultModel->clear();
    audioList->model->clear();
    emit projectCreated();
    update();
});

connect(openProjectAction, &QAction::triggered, this, [&]() {

    QString filePath = QFileDialog::getOpenFileName(this, "Open Project File",
QDir::homePath(), "Project Files (*.sfp)");
    if (!filePath.isEmpty()) {
        QFile file(filePath);
        if (!file.open(QIODevice::ReadOnly)) {

            msgWarning(L"Could not open file for reading.", L"Error");
            return; // Выходим, если файл не открылся
        }

        defaultModel->clear();

        projectFullPath = filePath;
        projectPath = QFile::FileInfo(filePath).absolutePath();

        QDir currDir(projectPath);
        projectDir = currDir;
        audioList->loadAudioFiles(projectDir.filePath("resources"));

        // Чтение количества дорожек
        int trackCount = 0;
        if (file.read(reinterpret_cast<char*>(&trackCount), sizeof(int)) !=
sizeof(int)) {

            QMessageBox msgBox(this);
            msgBox.setWindowTitle("Error");
            msgBox.setIcon(QMessageBox::Warning); // Установка иконки с
вопросительным знаком
            msgBox.setText("File is empty.");
            msgBox.setStandardButtons(QMessageBox::Yes | QMessageBox::No);

            // Применение стиля к QMessageBox
            msgBox.setStyleSheet(
                "QPushButton { background-color: " +
ProjectConfiguration::clDarkButton.name() + "; color: " +
ProjectConfiguration::clSidePanelText.name() + "; }"
                + "QPushButton:hover { background-color: " +
ProjectConfiguration::clDarkButtonSelected.name() + "; }"
            );

```

```

        msgBox.exec();
        file.close();
        return;
    }

    for (int i = 0; i < trackCount; i++) {
        AudioTrack *track = new AudioTrack();
        track->isTrackChanged = true;
        defaultModel->addTrack(track);
    }

    // Чтение данных токенов
    while (!file.atEnd()) {
        int tokenAudiofileID;
        double tokenStartPositionView;
        double tokenDurationView;
        double tokenRelativeStartTimeView;
        double tokenRelativeDurationView;
        int tokenAudioTrack;

        // Чтение данных из файла
        file.read((char*)&tokenAudiofileID, sizeof(tokenAudiofileID));
        file.read((char*)&tokenStartPositionView,
sizeof(tokenStartPositionView));
        file.read((char*)&tokenDurationView, sizeof(tokenDurationView));
        file.read((char*)&tokenRelativeStartTimeView,
sizeof(tokenRelativeStartTimeView));
        file.read((char*)&tokenRelativeDurationView,
sizeof(tokenRelativeDurationView));
        file.read((char*)&tokenAudioTrack, sizeof(tokenAudioTrack));

        if (tokenAudioTrack < 0 || tokenAudioTrack >= defaultModel-
>tracks.size()) {
            QMessageBox::warning(this, "Error", "Invalid token data (audio
track out of range).");
            break;
        }

        AudioToken* token = new AudioToken(tokenAudiofileID,
                                            tokenStartPositionView,
tokenDurationView,
                                            tokenAudioTrack, linker,
tokenRelativeStartTimeView, tokenRelativeDurationView);

        defaultModel->tracks[token->audioTrack]->tokens.append(*token);
    }

    file.close(); // Явно закрываем файл
}
emit projectCreated();
});

connect(saveProjectAction, &QAction::triggered, this, [&]() {
    if (projectFullPath.isEmpty()) {
        QMessageBox::warning(this, "Error", "No project file to save.");
        return;
    }

    QFile file(projectFullPath);
    if (!file.open(QIODevice::WriteOnly)) {
        QMessageBox::warning(this, "Error", "Could not open file for writing.");
        return;
    }

    // Запись количества дорожек
    int trackCount = defaultModel->tracks.size();
    file.write((char*)&trackCount, sizeof(int));

    for (AudioTrack* track : defaultModel->tracks) {
        for (const AudioToken& token : track->tokens) {

```

```

        file.write((char*)&token.audiofileID, sizeof(int));
        file.write((char*)&token.startPositionView, sizeof(double));
        file.write((char*)&token.durationView, sizeof(double));
        file.write((char*)&token.relativeStartTimeView, sizeof(double));
        file.write((char*)&token.relativeDurationView, sizeof(double));
        file.write((char*)&token.audioTrack, sizeof(int));
    }
}

file.close(); // Явно закрываем файл
QMessageBox::information(this, "Project Saved", "Project saved
successfully.");
});

connect(importAudioAction, &QAction::triggered, this, [&]() {
    QStringList filePaths = QFileDialog::getOpenFileNames(this, "Import Audio
Files", QDir::homePath(), "Audio Files (*.wav *.mp3)");
    if (!filePaths.isEmpty()) {
        for (const QString &filePath : filePaths) {
            QFile::copy(filePath, projectDir.filePath("resources/" +
QFileInfo(filePath).fileName()));
        }
        QMessageBox::information(this, "Audio Files Imported", "Audio files
imported to project resources.");
        audioList->loadAudioFiles(projectDir.filePath("resources"));
    }
});

connect(exportAudioAction, &QAction::triggered, this, [&]() {
    QString filePath = QFileDialog::getSaveFileName(this, "Export Audio File",
QDir::homePath(), "Audio Files (*.wav)");
    if (!filePath.isEmpty()) {
        QFile file(filePath);
        if (file.open(QIODevice::WriteOnly)) {
            AudioTrack *currTrack;
            QByteArray *res = new QByteArray();
            trackFrame->totalTrack = defaultModel->tracks[0];
            trackFrame->totalTrack->processAudioTrack();
            for (int i = 1; i < defaultModel->tracks.size(); i++) {
                currTrack = defaultModel->tracks[i];
                if (currTrack->tokens.isEmpty()) continue;

                if (currTrack->isTrackChanged)
                    currTrack->processAudioTrack();

                *res = trackFrame->mixWavFiles(trackFrame->totalTrack->trackData,
currTrack->trackData);
                trackFrame->totalTrack->trackData = res;
            }

            if (trackFrame->totalTrack == nullptr || trackFrame->totalTrack-
>trackData->isEmpty()) {
                QMessageBox::warning(this, "Error", "No data to write.");
                return;
            }

            QByteArray data = *(trackFrame->totalTrack->trackData);
            qint64 bytesWritten = file.write(data);
            if (bytesWritten == -1) {
                QMessageBox::warning(this, "Error", "Failed to write data to
file.");
            }
            file.close();
        } else {
            QMessageBox::warning(this, "Error", "Could not open file for
writing.");
        }
    }
});

```

```

connect(closeAction, &QAction::triggered, this, &MainWindow::close);

connect(helpAction, &QAction::triggered, this, [this]() {
    // Создаем диалоговое окно
    QDialog *helpDialog = new QDialog(this);
    helpDialog->setWindowTitle("Help");

    // Создаем QLabel с текстом
    QLabel *helpLabel = new QLabel(
        "МУЗЫКАЛЬНЫЙ РЕДАКТОР SOUND_FORGE:\n\n"
        "1. Для начала работы с программой необходимо создать проект нажав File ->
Create project и выбрав директорию где будет храниться проект\n"
        "2. Затем импортируйте исходные аудиофайлы - File -> Import audiofiles\n"
        "3. Добавьте одну или несколько аудиодорожек, нажав Add track\n"
        "4. Перетащите исходные аудиофайлы на аудиодорожки и отредактируйте их, как
захотите\n"
        "5. Экспортируйте готовый файл нажав File -> Export\n\n"

        "РЕДАКТИРОВАНИЕ СЭМПЛОВ\n"
        "1. Размещенные сэмплы можно перетаскивать по аудиодорожкам.\n"
        "2. Ненужные токены можно удалить с помощью ПКМ + Delete token\n"
        "3. Вы можете прослушать трек перед экспортом.\n"
        "4. Для редактирования конкретного сэмпла нажмите ПКМ + Edit token\n",
        helpDialog
    );

    // Настройка стиля
    helpDialog->setStyleSheet
    (
        "QDialog, QLabel { background-color: " +
ProjectConfiguration::clSidePanel.name() + "; color: " +
ProjectConfiguration::clSidePanelText.name() + "; }"
        + "QLabel {font-size: 12pt; font-family: Arial; }"
        + "QPushButton { background-color: " +
ProjectConfiguration::clDarkButton.name() + "; color: " +
ProjectConfiguration::clSidePanelText.name() + "; }"
        + "QPushButton:hover { background-color: " +
ProjectConfiguration::clDarkButtonSelected.name() + "; }"
    );

    helpLabel->setStyleSheet("border: none;"); // Убираем рамку у QLabel

    // Создаем кнопку для закрытия диалога
    QPushButton *closeButton = new QPushButton("Close", helpDialog);
    connect(closeButton, &QPushButton::clicked, helpDialog, &QDialog::accept);

    // Устанавливаем вертикальный layout
    QVBoxLayout *layout = new QVBoxLayout(helpDialog);
    layout->addWidget(helpLabel);
    layout->addWidget(closeButton);

    // Устанавливаем layout для диалога
    helpDialog->setLayout(layout);

    // Показываем диалог
    helpDialog->exec();
});

connect(aboutAction, &QAction::triggered, this, [this]() {
    // Создаем диалоговое окно
    QDialog *aboutDialog = new QDialog(this);
    aboutDialog->setWindowTitle("About");

    // Создаем QLabel с текстом
    QLabel *aboutLabel = new QLabel(
        "О ПРИЛОЖЕНИИ:\n\n"
        "SoundForge – это мощный аудиоредактор, который предоставляет
пользователям необходимые инструменты для работы с аудиофайлами.\n"
        "SoundForge предлагает широкий спектр возможностей для редактирования
аудиофайлов и обработки звука.\n\n"
    );

```



```

        "Разработчик: Наривончик Александр, гр. 351004\n",
        aboutDialog
    );

    // Настройка стиля
    aboutDialog->setStyleSheet
    (
        "QDialog, QLabel { background-color: " +
ProjectConfiguration::clSidePanel.name() + "; color: " +
ProjectConfiguration::clSidePanelText.name() + "; }"
        + "QLabel {font-size: 12pt; font-family: Arial; }"
        + "QPushButton { background-color: " +
ProjectConfiguration::clDarkButton.name() + "; color: " +
ProjectConfiguration::clSidePanelText.name() + "; }"
        + "QPushButton:hover { background-color: " +
ProjectConfiguration::clDarkButtonSelected.name() + "; }"
    );

    aboutLabel->setStyleSheet("border: none;"); // Убираем рамку у QLabel

    // Создаем кнопку для закрытия диалога
    QPushButton *closeButton = new QPushButton("Close", aboutDialog);
    connect(closeButton, &QPushButton::clicked, aboutDialog, &QDialog::accept);

    // Устанавливаем вертикальный layout
    QVBoxLayout *layout = new QVBoxLayout(aboutDialog);
    layout->addWidget(aboutLabel);
    layout->addWidget(closeButton);

    // Устанавливаем layout для диалога
    aboutDialog->setLayout(layout);

    // Показываем диалог
    aboutDialog->exec();
});

}

void MainWindow::onProjectCreated() {
    isProjectCreated = true;
}

```